UNIVERSIDADE DO ESTADO DO AMAZONAS – UEA

ESCOLA SUPERIOR DE TECNOLOGIA – EST

CURSO DE ENGENHARIA ELÉTRICA

LUANNE LYS DE SOUZA OLIVEIRA

PLATAFORMA E BIBLIOTECA VIRTUAL DE SISTEMAS E PROCESSOS DE CONTROLE PARA APOIO AO DESENVOLVIMENTO DE LABORATÓRIOS VIRTUAIS NO ENSINO DA ENGENHARIA ELÉTRICA E ELETRÔNICA

Manaus

2024

LUANNE LYS DE SOUZA OLIVEIRA

PLATAFORMA E BIBLIOTECA VIRTUAL DE SISTEMAS E PROCESSOS DE CONTROLE PARA APOIO AO DESENVOLVIMENTO DE LABORATÓRIOS VIRTUAIS NO ENSINO DA ENGENHARIA ELÉTRICA E ELETRÔNICA

Projeto de pesquisa desenvolvido durante a disciplina de Trabalho de Conclusão de Curso II e apresentado à banca avaliadora do Curso de Engenharia Elétrica da Escola Superior de Tecnologia da Universidade do Estado do Amazonas, como pré-requisito para a obtenção do título de Engenheiro Eletricista.

Orientação: Daniel Guzmán del Río, Dr.

Manaus

2024

Universidade do Estado do Amazonas – UEA
Escola Superior de Tecnologia - EST

Reitor:
André Luiz Nunes Zogahib
Vice-Reitor:
Kátia do Nascimento Couceiro
Diretor da Escola Superior de Tecnologia:
Jucimar Maia da Silva Júnior
Coordenador do Curso de Engenharia Elétrica:
Jozias Parente de Oliveira

Banca Avaliadora composta por: *Data da defesa: 24/07/2024.*
Prof. Daniel Guzmán del Río, Dr. (Orientador)
Prof. Antonio Luiz Alencar Pantoja, Dr. (Avaliador 1)
Prof. Edry Antonio Garcia Cisneros, Dr. (Avaliador 2)

CIP – Catalogação na Publicação

Oliveira, Luanne Lys de Souza

Plataforma e Biblioteca Virtual de sistemas e processos de controle para apoio ao desenvolvimento de laboratórios virtuais no ensino da Engenharia Elétrica e Eletrônica./ Luanne Lys de Souza Oliveira; orientado por Daniel Guzmán del Río. – Manaus: 2024.
83 p.: il.

Trabalho de Conclusão de Curso (Graduação em Engenharia Elétrica). Universidade do Estado do Amazonas, 2024.

1. Sistemas de Controle. 2. Biblioteca Virtual. 3. Matlab. 4. App Designer.
I. del Río, Daniel Guzmán.

LUANNE LYS DE SOUZA OLIVEIRA

PLATAFORMA E BIBLIOTECA VIRTUAL DE SISTEMAS E PROCESSOS DE CONTROLE PARA APOIO AO DESENVOLVIMENTO DE LABORATÓRIOS VIRTUAIS NO ENSINO DA ENGENHARIA ELÉTRICA E ELETRÔNICA

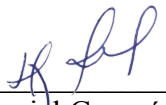
Projeto de pesquisa desenvolvido durante a disciplina de Trabalho de Conclusão de Curso II e apresentado à banca avaliadora do Curso de Engenharia Elétrica da Escola Superior de Tecnologia da Universidade do Estado do Amazonas, como pré-requisito para a obtenção do título de Engenheiro Eletricista.

Nota obtida: 10 pontos

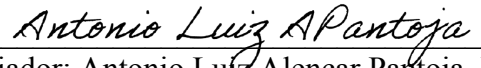
Data da defesa: 24/07/2024

Área de concentração: Sistemas de Controle

BANCA EXAMINADORA



Orientação: Daniel Guzmán del Río, Dr.



Avaliador: Antonio Luiz Alencar Pantoja, Dr.



Avaliador: Edry Antonio Garcia Cisneros, Dr.

Manaus

2024

DEDICATÓRIA

Dedico este trabalho à minha família, pelo amor e apoio constante em todas as etapas da minha vida.

AGRADECIMENTOS

A realização deste Trabalho de Conclusão de Curso representa o fim de um ciclo, de uma jornada repleta de desafios, aprendizados e crescimento. Não poderia ter chegado até aqui sem o apoio de pessoas especiais que, direta ou indiretamente, contribuíram para este sucesso.

Primeiramente, gostaria de expressar minha profunda gratidão à minha mãe, Antonia Eliceia de Souza, cuja dedicação, amor e incentivo aos estudos foram fundamentais para que eu pudesse chegar até este momento. Suas palavras de encorajamento e sua presença constante me deram forças para superar os obstáculos ao longo deste caminho.

Às minhas irmãs, Lauren Souza e Lilian Souza, pelo carinho, apoio e compreensão. Os momentos leves e risadas facilitaram muito a minha trajetória, obrigada por estarem ao meu lado.

Aos amigos e colegas de classe, agradeço a colaboração e troca de experiências. Juntos, superamos muitos desafios e comemoramos diversas conquistas.

Ao meu professor, Dr. Daniel Guzmán del Río, por sua orientação e conhecimentos compartilhados. Seu suporte e suas contribuições foram fundamentais durante o desenvolvimento do trabalho.

Por fim, muito obrigada a todos que fizeram parte da minha trajetória acadêmica.

RESUMO

O trabalho apresenta o desenvolvimento de uma Biblioteca Virtual de Sistemas de Controle destinada aos estudantes de engenharia elétrica e eletrônica da Universidade do Estado do Amazonas (UEA). A proposta é oferecer um aplicativo interativo para auxiliar os alunos no processo de aprendizagem, e assim, incentivar o desenvolvimento de laboratórios virtuais no ensino de outras disciplinas dos cursos de Engenharia. O referencial teórico inclui a revisão bibliográfica de sistemas de controle, e a descrição do ambiente virtual utilizado para desenvolver a plataforma, o App Designer, ferramenta do Matlab. Em seguida, mostrou-se a abordagem de implementação do projeto e a versão final do aplicativo, composto por sete interfaces gráficas. Após a pesquisa de aceitação realizada com os estudantes de Sistemas de Controle I, do curso de engenharia elétrica, constatou-se que a biblioteca virtual é uma ferramenta interativa com design, layout, navegação e usabilidade bem avaliados. Além disso, a acessibilidade e flexibilidade foram obtidas, pois independente do horário e localização, os estudantes podem acessar os experimentos.

Palavras-chave: Sistemas de Controle, Biblioteca Virtual, Matlab, App Designer.

ABSTRACT

This final project presents the development of a Virtual Library of Control Systems for electrical and electronic engineering students at the State University of Amazonas (UEA). The proposal is to offer an interactive application to assist students in their learning process, thus encouraging the development of virtual laboratories for teaching other engineering courses. The theoretical framework includes a bibliographic review of control systems and a description of the virtual environment used to develop the platform, the App Designer, a Matlab tool. Then, the project implementation approach and the final version of the application, consisting of seven graphical interfaces, were presented. After an acceptance survey conducted with students of Control Systems I, from the electrical engineering course, it was found that the virtual library is an interactive tool with well-evaluated design, layout, navigation, and usability. Additionally, accessibility and flexibility were achieved, allowing students to access the experiments regardless of time and location.

Keywords: Control Systems, Virtual Library, Matlab, App Designer.

LISTA DE FIGURAS

Figura 1: Fluxograma de contextualização do trabalho. 16
Figura 2: Estrutura básica dos sistemas de controle. 18
Figura 3: Esquema geral de sistemas de malha aberta e malha fechada 19
Figura 4: Curva de resposta exponencial a entrada degrau unitário. 20
Figura 5: Resposta a rampa unitária para sistemas de primeira ordem. 21
Figura 6: Classificação das respostas de sistemas de segunda ordem. 23
Figura 7: Representação e análise de um sistema estável. 24
Figura 8: Representação e análise de um sistema instável. 25
Figura 9: Configurações de polos e zeros de malha aberta e os correspondentes lugares das raízes. 26
Figura 10: Determinação das margens de ganho e fase. 28
Figura 11: Exemplo de contorno de Nyquist para determinar a estabilidade. 30
Figura 12: Diagrama de blocos de um Controlador PID. 30
Figura 13: Respostas típicas de controladores. 32
Figura 14: Divisão do App Designer: Seção *Design View*. 34
Figura 15: Divisão do App Designer: Seção *Code View*. 35
Figura 16: Menu *Component Browser* do controle deslizante *(slider)*. 35
Figura 17: Resposta a eventos nos componentes gráficos. 36
Figura 18: *Callbacks* disponíveis no componente *slider*. 37
Figura 19: Criação de *callbacks*, *functions* e *properties* no Editor do *Code View*. 39
Figura 20: Menu *Code Browser* do *Code View*. 39
Figura 21: Etapas para implementação da Biblioteca Virtual. 41
Figura 22: Página inicial do App Designer. 42
Figura 23: Página inicial da Biblioteca Virtual. 56
Figura 24: Guia “Simulação” da Interface Gráfica 1. 57
Figura 25: Guia “Exemplo” da Interface Gráfica 1. 57
Figura 26: Guia “Simulação” da Interface Gráfica 2. 58
Figura 27: Guia “1 - Exemplo” da Interface Gráfica 2. 59
Figura 28: Guia “2 - Exemplo” da Interface Gráfica 2. 59
Figura 29: Guia “Simulação” da Interface Gráfica 3. 60
Figura 30: Guia “1 - Exemplo” da Interface Gráfica 3. 61

Figura 31: Guia “2 - Exemplo” da Interface Gráfica 3. 61
Figura 32: Guia “Simulação” da Interface Gráfica 4. 62
Figura 33: Guia “1 - Exemplo” da Interface Gráfica 4. 63
Figura 34: Guia “2 - Exemplo” da Interface Gráfica 4. 63
Figura 35: Guia “Simulação” da Interface Gráfica 5. 64
Figura 36: Guia “Exemplo” da Interface Gráfica 5. 64
Figura 37: Guia “Simulação” da Interface Gráfica 6. 65
Figura 38: Guia “1 - Exemplo” da Interface Gráfica 6. 66
Figura 39: Guia “2 - Exemplo” da Interface Gráfica 6. 66
Figura 40: Guia “Simulação” da Interface Gráfica 7. 67
Figura 41: Guia “1 - Exemplo” da Interface Gráfica 7. 68
Figura 42: Guia “2 - Exemplo” da Interface Gráfica 7. 68
Figura 43: Guia “Bibliografia” presente nas 7 Interfaces. 69
Figura 44: Guia de compilação do aplicativo Desktop. 70
Figura 45: Arquivos instalados para a execução do app e tamanho especificado de 991MB. 71
Figura 46: Localização do arquivo principal e opção de atalho na área de trabalho. 71
Figura 47: Afirmação “A Biblioteca Virtual é de fácil navegação e utilização”. 72
Figura 48: Afirmação “A interface e design do laboratório virtual é intuitiva e agradável”. 72
Figura 49: Afirmação “Ótimos recursos disponíveis para simulações e interatividade”. 73
Figura 50: Afirmação “A Biblioteca virtual ajudou você a entender melhor os conceitos de Sistemas de Controle”. 73
Figura 51: Afirmação “Qual a sua satisfação geral com a biblioteca virtual?”. 73
Figura 52: Pergunta “Você recomendaria esta biblioteca virtual para outros alunos?”. 74

LISTA DE TABELAS

Tabela 1: Passo a passo do traçado do lugar geométrico das raízes. 25
Tabela 2: Lista de componentes gráficos utilizados na implementação da Biblioteca Virtual. 43
Tabela 3: Comandos utilizados na implementação da Biblioteca Virtual. 44
Tabela 4: Recursos utilizados na implementação da Interface 1. 47
Tabela 5: Recursos utilizados na implementação da Interface 2. 49
Tabela 6: Recursos utilizados na implementação da Interface 3. 50
Tabela 7: Recursos utilizados na implementação da Interface 4. 51
Tabela 8: Recursos utilizados na implementação da Interface 5. 52
Tabela 9: Recursos utilizados na implementação da Interface 6. 53
Tabela 10: Recursos utilizados na implementação da Interface 7. 54

LISTAS DE EQUAÇÕES

Equação 1: Equação geral da função de transferência. 19
Equação 2: Função de transferência de sistemas de primeira ordem. 20
Equação 3: Função de transferência ao aplicar a entrada degrau unitário. 20
Equação 4: Curva de resposta do sistema a entrada degrau. 20
Equação 5: Função de transferência ao aplicar a entrada rampa. 21
Equação 6: Curva de resposta do sistema a entrada rampa. 21
Equação 7: Representação do erro atuante. 21
Equação 8: Função de transferência de sistemas de segunda ordem. 22
Equação 9: Resposta total do sistema. 23
Equação 10: Representação do Controlador Proporcional. 31
Equação 11: Representação do Controlador Proporcional Integral. 31
Equação 12: Representação do Controlador Proporcional Derivativo. 32
Equação 13: Representação do Controlador PID. 32
Equação 14: Função de transferência do controle do nível de líquido. 47
Equação 15: Função de transferência do sistema massa mola amortecedor. 48
Equação 16: Função de transferência do do circuito elétrico com amplificador operacional. 50
Equação 17: Sistema para simulação do LGR. 51
Equação 18: Sistema para simulação do Diagrama de Bode. 52
Equação 19: Representação do sistema para simulação do Diagrama de Nyquist. 53

LISTA DE ABREVIATURAS, SIGLAS E SÍMBOLOS

dB - Decibel

GUI - Interface Gráfica de Usuário (*Graphical User Interface)*

Kp - Ganho Proporcional

Ki - Ganho Integral

Kd - Ganho Derivativo

LGR – Lugar Geométrico das Raízes

LV – Laboratório Virtual

P – Proporcional

PI – Proporcional Integral

PD – Proporcional Derivativo

PID - Proporcional, Integral e Derivativo

UI – Interface de usuário (*User Interface*)

SUMÁRIO

INTRODUÇÃO 15
1. REFERENCIAL TEÓRICO 17
1.1 Conceitos Básicos de Sistemas de Controle 17
1.1.1 Definição de Sistemas de Controle e seus componentes 17
1.1.2 Configurações de Sistemas de Controle: Malha aberta e Malha Fechada 18
1.1.3 Função de Transferência 19
1.1.4 Resposta temporal de Sistemas de Primeira Ordem 19
1.1.5 Resposta dinâmica de Sistemas de Segunda Ordem 22
1.1.6 Estabilidade de Sistemas de Controle 23
1.1.7 Lugar Geométrico das Raízes (LGR) 25
1.1.8 Diagrama de Bode 27
1.1.9 Diagrama de Nyquist 28
1.1.10 Ações básicas de controle: Proporcional, Integral e Derivativo 30
1.2 Recursos computacionais para a implementação da Biblioteca Virtual 33
1.2.1 Software Matlab 33
1.2.2 App Designer: Definição 33
1.2.3 Estrutura do App Designer 34
1.2.4 *Design View*: Características e propriedades 35
1.2.5 *Code View*: Características e propriedades 36
2. METODOLOGIA 40
3. DESENVOLVIMENTO DA BIBLIOTECA VIRTUAL 42
3.1 Comandos e componentes gráficos utilizados no App Designer 42
3.2 Layout do aplicativo 45
3.3 Descrição das Interfaces Gráficas 47
3.4 Apresentação das Interfaces Gráficas 56
3.5 Compilação e instalação da Biblioteca Virtual 70
4. PESQUISA DE ACEITAÇÃO 72
5. CONCLUSÃO 75
6. REFERÊNCIAS BIBLIOGRÁFICAS 76
7. FONTES CONSULTADAS 78
ANEXO A – CÓDIGOS DA BIBLIOTECA VIRTUAL 79
ANEXO B – DICAS PARA SIMULAÇÕES NA BIBLIOTECA VIRTUAL 83

INTRODUÇÃO

A busca por novas metodologias de ensino, com o propósito de melhorar o processo de desenvolvimento das habilidades técnicas e comportamentais dos alunos é constante nas universidades brasileiras. Dessa maneira, a implantação de novas abordagens de ensino-aprendizagem tem ocorrido frequentemente nas instituições que primam pela qualidade de seu ensino e pelo atendimento aos seus alunos (Agostinho, 2022, p.02).

Outro fator que também incentiva e impulsiona esta demanda é a necessidade crescente do mercado de trabalho por profissionais com habilidades multidisciplinares. Para Sutili e Ranieri (2022), o mercado atual, altamente globalizado e transformado, requer um engenheiro que tenha uma formação baseada em preceitos como: empreendedorismo, trabalho em equipes multidisciplinares, desenvolvimento e administração de projetos, liderança e capacidade de aprendizado independente.

Estes princípios e competências são abordados, incentivados e desenvolvidos nas denominadas metodologias ativas. Essas são definidas como estratégias, técnicas, abordagens e perspectivas de aprendizagem individual e colaborativa que envolvem e engajam os estudantes no desenvolvimento de projetos e/ou atividades práticas. Nos contextos em que são adotadas, o aprendiz é visto como um sujeito ativo, que deve participar de forma intensa de seu processo de aprendizagem, enquanto reflete sobre aquilo que está fazendo (Filatro; Cavalcanti, 2018, p. 27).

Em vista disso, nota-se que os laboratórios e bibliotecas virtuais apresentam-se como formas de metodologia ativa que utilizam tecnologias digitais para aprimorar a aquisição de conhecimentos e promover o aprendizado ágil e eficiente dos alunos. Logo, apresentam-se como uma nova proposta didática em disciplinas de graduação de engenharia, e podem representar um impacto significativo na curva de aprendizagem dos alunos (Vidotto; Tarouco; Silva; Faraco, 2022).

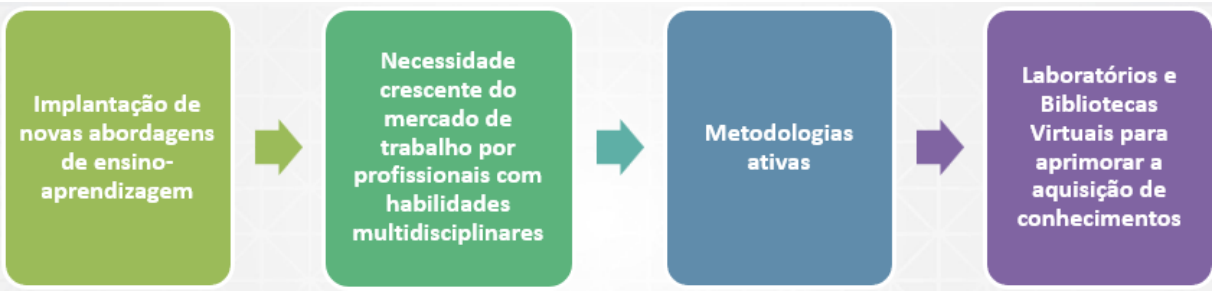
Em trabalhos recentes, Schmitz, Nogueira e Nervis (2021) avaliaram qualitativamente o Laboratório Virtual de Aprendizagem na Engenharia Civil para o conhecimento básico de solos, como: índices físicos, granulometria e índices de consistência. E destacaram que os laboratórios virtuais podem colaborar positivamente e atuar como instrumentos auxiliares no processo de aprendizagem dos alunos.

A pesquisa de De Matheucci et al. (2021), realizou uma análise pedagógica de um laboratório virtual de ensino de Engenharia como apoio a um experimento real de hidráulica.

E os resultados da investigação verificaram que o LV tem caráter didático, o aluno tem autonomia para utilizar o recurso a qualquer momento, seguir o seu ritmo e ser protagonista (Vidotto; Tarouco; Silva; Faraco, 2022).

Abaixo encontra-se o fluxograma com o resumo e contextualização dos motivos para a realização do trabalho.

Figura 1: Fluxograma de contextualização do trabalho.



Fonte: (Própria, 2024).

Diante desses argumentos, a proposta deste trabalho é apresentar uma plataforma e biblioteca virtual, desenvolvida no Matlab, aos alunos de engenharia elétrica e eletrônica, com a finalidade de simular e compreender o comportamento de processos de controle, colocando-os assim como sujeitos ativos no seu processo de aprendizagem, e consequentemente promover e ajudar na formação dessas novas competências que estão sendo cada vez mais exigidas no mercado de trabalho.

A implementação do projeto justifica-se por diversas razões. Primeiramente, facilitar a aplicação de conceitos de sistemas de controle na formação de estudantes de graduação, ao integrar os conteúdos explicados em sala de aula com o software Matlab através de simulações. Outro aspecto importante é a acessibilidade e flexibilidade, que permite aos estudantes, independentemente de sua localização e horário, acessar aos experimentos e práticas de laboratório. Por fim, a variedade de experimentos e simulações possíveis no ambiente virtual permite a reprodução de uma ampla variedade de processos em sistemas de controle, os quais poderiam ser caros e difíceis de replicar em um laboratório físico.

1. REFERENCIAL TEÓRICO

1.1 Conceitos Básicos de Sistemas de Controle

Previamente a implementação do projeto, é necessário abordar e definir os conceitos fundamentais de sistemas de controle. Dessa forma, o aluno estará familiarizado com os termos e definições dos processos, para assim, ser capaz de interpretar e identificar com maior facilidade a resposta e estabilidade dos sistemas propostos.

1.1.1 Definição de Sistemas de Controle e seus componentes

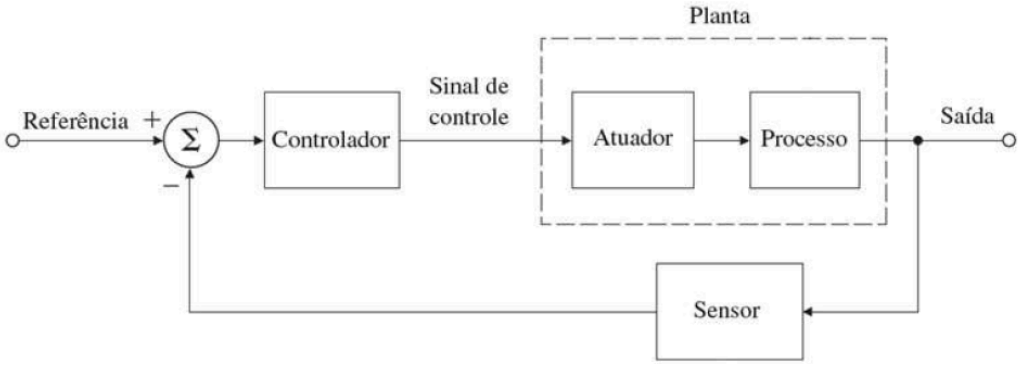
Os sistemas de controle estão presentes em diversos processos do nosso dia a dia. Incontáveis são as suas aplicações e benefícios no campo da engenharia e indústria, como por exemplo, no controle do nível de líquido de reservatórios, regulação da temperatura e pressão em ambientes, controle de velocidade de motores elétricos, robótica ou até mesmo para o funcionamento bem-sucedido de veículos espaciais.

Com o intuito de otimizar o desempenho e garantir a estabilidade, define-se que um sistema de controle consiste em subsistemas e processos construídos com objetivo de se obter uma saída desejada com um desempenho desejado, dada uma entrada especificada (NISE, 2012, p.28). Dessa forma, a composição básica de um sistema de controle é mostrado na figura 2, com os seguintes componentes:

- Processo: sistema físico que deve ser regulado ou gerenciado.
- Variável manipulada: entrada do sistema, variável na qual o controlador atua, com o intuito forçar a variável controlada a seguir o sinal de referência.
- Variável controlada: é a saída de uma planta ou processo que o sistema está regulando para obter a resposta transitória desejada, assegurar a estabilidade e minimizar o erro em regime permanente.
- Atuadores: dispositivos que atuam fisicamente no processo ao alterar o estado ou comportamento de uma variável, transformam outros tipos de energia em energia mecânica. Exemplos: válvulas, propulsores, bombas etc.
- Controlador: subsistema que recebe o sinal de referência, e emite o sinal de controle para a planta, com o objetivo manter a variável controlada dentro do limite desejado.
- Sensor: dispositivos que medem a variável controlada e a convertem em um sinal elétrico ou digital que pode ser lido pelo controlador.

- Planta: parte do sistema que inclui o processo físico e os atuadores. Executa as ações ordenadas pelo controlador.

Figura 2: Estrutura básica dos sistemas de controle.



Fonte: (Almeida, 2021, p.13).

1.1.2 Configurações de Sistemas de Controle: Malha aberta e Malha Fechada

Os chamados sistemas de controle de malha aberta são aqueles em que o sinal de saída não exerce nenhuma ação de controle no sistema. Isso quer dizer que, a variável controlada não é medida, nem realimentada para comparação com a entrada. Portanto, cada entrada de referência corresponde uma condição fixa de operação (Ogata, 2010, p.07).

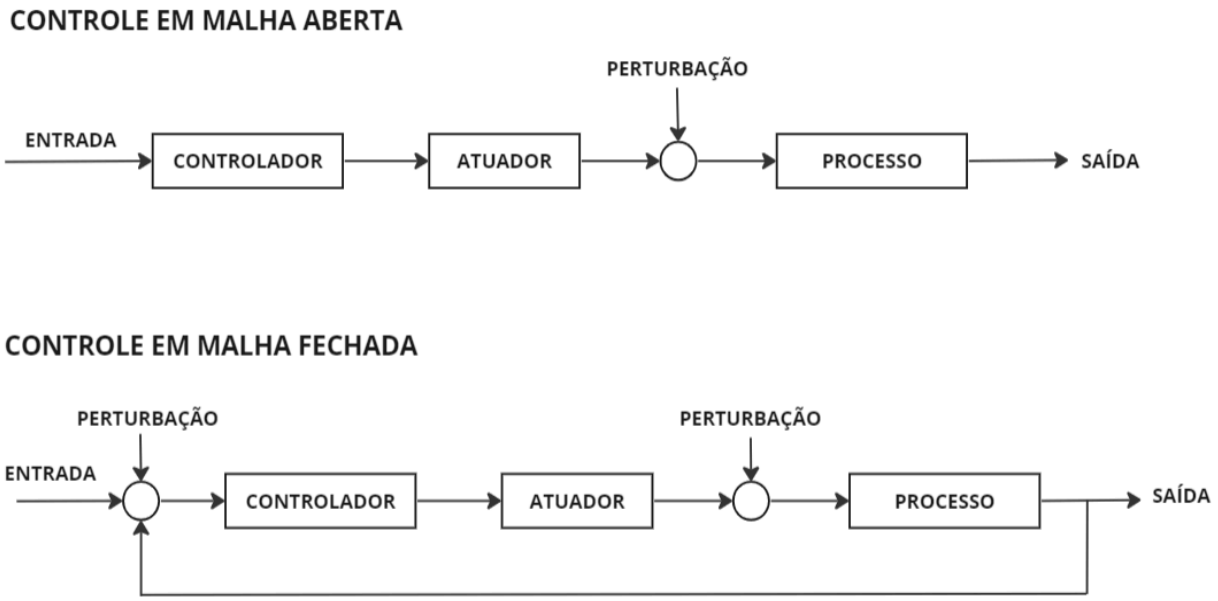
Dessa maneira, a precisão do sistema depende de uma calibração. Na presença de distúrbios, um sistema de controle de malha aberta não vai executar a tarefa desejada. Na prática, ele poderá ser utilizado se a relação entre a entrada e a saída for conhecida e se não houver nenhum distúrbio interno ou externo (Ogata, 2010, p.07).

Outra configuração são os sistemas de controle de malha fechada, também conhecidos como sistemas de controle com retroalimentação, que monitoram continuamente o valor da variável controlada e o subtraem do valor da variável de entrada. De modo que, o sinal de erro atuante realimenta o controlador, minimizando assim a porcentagem do erro e ajustando o sinal de saída ao valor próximo do setpoint desejado.

Desta maneira, os sistemas em malha fechada possuem a vantagem óbvia de apresentar uma exatidão maior que os sistemas em malha aberta. Estes são menos sensíveis a ruídos, perturbações e alterações do ambiente. A resposta transitória e os erros em regime permanente podem ser controlados de modo mais conveniente e com maior flexibilidade nos sistemas em malha fechada, frequentemente pelo simples ajuste de um ganho (amplificação) na malha e, algumas vezes, ajustando-se o projeto do controlador. Por outro lado, os sistemas

em malha fechada são mais complexos e caros que sistemas em malha aberta (Nise, 2012, p.37).

Figura 3: Esquema geral de sistemas de malha aberta e malha fechada.



Fonte: (Própria, 2024).

1.1.3 Função de Transferência

A função de transferência de um sistema ou processo, representado por uma equação diferencial linear invariante no tempo, é definida como a relação entre a transformada de Laplace da saída e a transformada de Laplace da entrada, admitindo-se todas as condições iniciais nulas (Ogata, 2010, p.12), como pode ser visto na equação 1.

Equação 1: Equação geral da função de transferência.

$$Função\ de\ Transferência\ = G(s) = \frac{L[saída]}{L[entrada]} \quad (condições\ iniciais\ nulas)$$

$$= \frac{Y(s)}{X(s)} = \frac{b_0 s^m + b_1 s^{m-1} + \ldots + b_{m-1} s + b_m}{a_0 s^n + a_1 s^{n-1} + \ldots + a_{n-1} s + a_n}$$

Por meio da função de transferência, é possível expressar a dinâmica de um sistema através de uma equação algébrica em s. Esta modelagem ajuda a analisar o comportamento, estabilidade e resposta em frequência do processo.

1.1.4 Resposta temporal de Sistemas de Primeira Ordem

Sistemas de primeira ordem possuem um único polo e são especificamente representados por equações diferenciais ordinárias de primeira ordem. Fisicamente,

podem modelar diversos fenômenos, como circuitos RC, sistemas de fluxo de líquidos, sistemas térmicos, entre outros semelhantes. A sua função de transferência é dada por:

Equação 2: Função de transferência de sistemas de primeira ordem.

$$G(s) = \frac{K}{\tau.s+1}$$

Onde K = Ganho estático e τ = Constante de tempo.

Quando estes sistemas são submetidos a entrada de teste degrau unitário, obtém-se a seguinte expressão:

Equação 3: Função de transferência ao aplicar a entrada degrau unitário.

$$G(s) = \frac{K}{\tau.s+1}.\frac{1}{s}$$

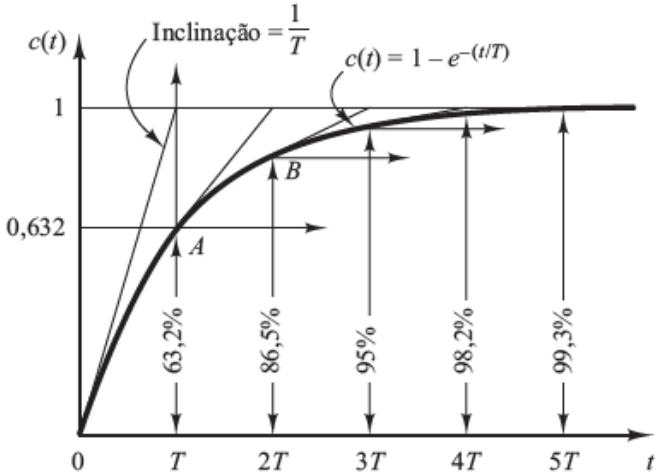
Ao aplicar a transformada inversa de Laplace, considerando o ganho K unitário, verifica-se a equação 4 de resposta:

Equação 4: Curva de resposta do sistema a entrada degrau.

$$g(t) = 1 - e^{-\frac{t}{\tau}}$$

Sendo caracterizada por uma curva exponencial, que eventualmente atinge o valor unitário, conforme mostrado na figura 4. Um aspecto importante é o termo $\frac{1}{\tau}$, denominado constante de tempo da resposta, que indica o intervalo necessário para a saída atingir 63,2% do seu valor final. Este termo também está diretamente relacionado à velocidade de resposta do sistema, uma constante de tempo maior prolonga o regime transiente, o que resulta em sistemas mais lentos.

Figura 4: Curva de resposta exponencial a entrada degrau unitário.



Fonte: (Ogata, 2010, p.148).

Outras especificações importantes da resposta transitória são o tempo de subida (Tr), e

o tempo de acomodação (Ts). O tempo de subida é definido como o tempo necessário para que a forma de onda vá de 0,1 a 0,9 de seu valor final. E o tempo de acomodação é definido como o tempo para que a resposta alcance e fique em uma faixa 2% em torno de seu valor final (Nise, 2012, p.251).

Em relação a resposta dos sistemas de primeira ordem a entrada de teste rampa, obtém-se a equação 5:

Equação 5: Função de transferência ao aplicar a entrada rampa.

$$G(s) = \frac{K}{\tau.s+1} . \frac{1}{s^2}$$

Ao aplicar a transformada inversa de Laplace, verifica-se a seguinte expressão:

Equação 6: Curva de resposta do sistema a entrada rampa.

$$g(t) = t - \tau - \tau . e^{-\frac{t}{\tau}}$$

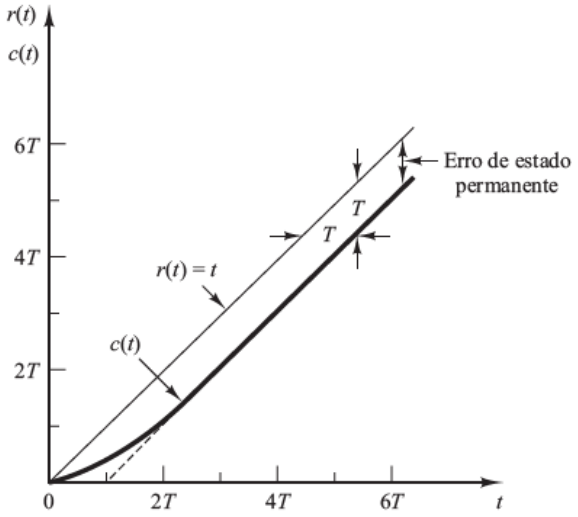
Onde uma propriedade importante é a representação do sinal de erro atuante, como mostrado na equação 7:

Equação 7: Representação do erro atuante.

$$e(t) = \tau\left(1 - e^{-\frac{t}{\tau}}\right)$$

À medida que $e^{-\frac{t}{\tau}}$ aproxima-se de zero, o sinal de erro atuante tende á τ. Portanto como pode-se observar na figura 5 abaixo, quanto menor a constante de tempo, menor será o erro estacionário do sistema a entrada rampa (Ogata, 2010).

Figura 5: Resposta a rampa unitária para sistemas de primeira ordem.



Fonte: (Ogata, 2010, p.149).

1.1.5 Resposta dinâmica de Sistemas de Segunda Ordem

Sistemas de controle de segunda ordem são definidos como modelos representados por uma equação diferencial de segunda ordem, o que significa que possuem dois polos. O sistema massa mola amortecedor, o modelo de um motor DC e um circuito elétrico com amplificador operacional são exemplos de sistemas de segunda ordem.

A equação geral da função de transferência desses sistemas, mostrado na equação 8, é representada através de duas grandezas, que são utilizadas para descrever as características de respostas transitórias: frequência natural (ωn) e fator de amortecimento (ξ). Sendo a frequência natural definida como a frequência de oscilação do sistema sem amortecimento. Já o fator de amortecimento, é fruto da necessidade de descrever quantitativamente a oscilação amortecida independentemente da escala de tempo, sendo definido como a razão entre a frequência de decaimento exponencial e a frequência natural (Nise, 2012, p.258).

Equação 8: Função de transferência de sistemas de segunda ordem.

$$G(s)\frac{\omega n^2}{s^2+2\xi.\omega n.s+\omega n^2}$$

Quando comparados à simplicidade de um sistema de primeira ordem, sistemas de segunda ordem exibem uma ampla variedade de respostas que devem ser analisadas e descritas. Enquanto a variação de um parâmetro de sistemas de primeira ordem simplesmente altera a velocidade da resposta, as variações nos parâmetros de um sistema de segunda ordem modificam a forma da resposta (Nise, 2012, p.252). Estas podem ser

classificadas em quatro tipos, observados na figura 6: não amortecida, subamortecida, superamortecida e criticamente amortecida. Suas respectivas características são descritas abaixo:

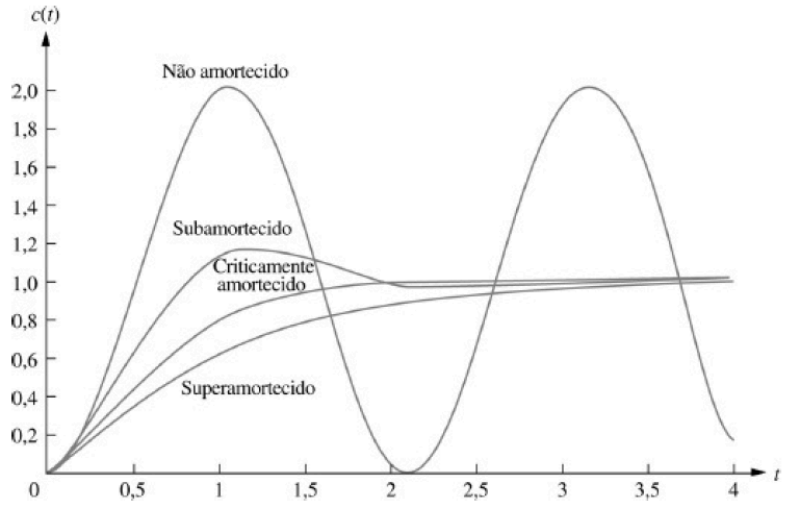
1. *Resposta não amortecida*
   - Coeficiente de amortecimento $\xi = 0$
   - Dois polos imaginários em $\pm j\omega$
   - O sistema oscila indefinidamente com amplitude constante, sem qualquer amortecimento.

2. *Resposta subamortecida*
   - Coeficiente de amortecimento $0 < \xi < 1$
   - Dois polos complexos conjugados $\sigma \pm j\omega$
   - O sistema apresenta oscilações cuja amplitude decresce exponencialmente com o tempo, antes de estabilizar-se no valor final.
   - Parâmetros associados: Tempo de subida (T_r), instante de pico (T_p), ultrapassagem percentual (%UP) e tempo de acomodação (T_s).

3. *Resposta superamortecida*
   - Coeficiente de amortecimento $\xi > 1$
   - Dois polos reais $\sigma 1$ e $\sigma 2$
   - O sistema retorna à posição de equilíbrio sem oscilações, de forma mais lenta quando comparado ao caso criticamente amortecido.

4. *Resposta criticamente amortecida*
   - Coeficiente de amortecimento $\xi = 1$
   - Dois polos reais e iguais
   - O sistema retorna o mais rápido à posição de equilíbrio, e a partir do valor de ξ não há mais oscilações.

Figura 6: Classificação das respostas de sistemas de segunda ordem.



Fonte: (Nise, 2012, p.256).

1.1.6 Estabilidade de Sistemas de Controle

A estabilidade é um conceito importante, que determina se o comportamento de um processo permanecerá controlado e previsível ao longo do tempo ou se apresentará respostas instáveis e indesejáveis. De acordo com Nise (2012): "a resposta total de um sistema é a soma das respostas forçada e natural" como mostrada na equação abaixo:

Equação 9: Resposta total do sistema.

$$c(t) = c_{forçada}(t) + c_{natural}(t)$$

Diante disso, três conceitos fundamentais são apresentados por Nise (2012):

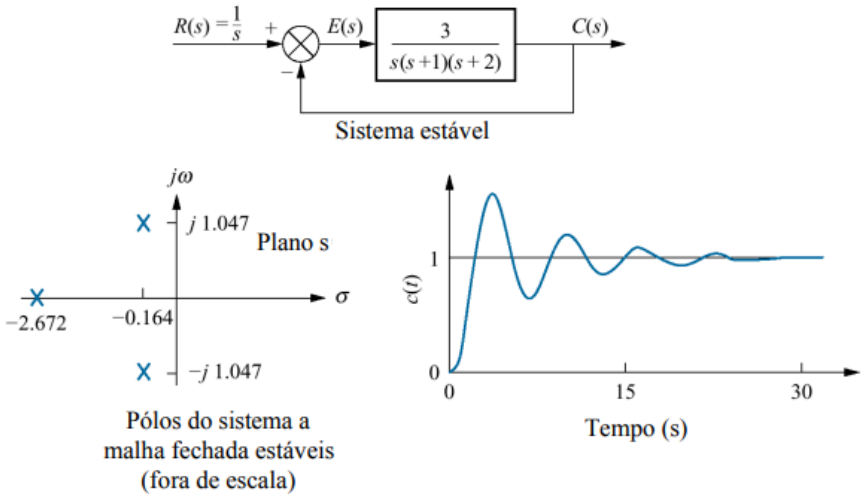
- Um sistema linear invariante no tempo é estável se a resposta natural tende a zero à medida que o tempo tende a infinito.
- Um sistema linear invariante no tempo é instável se a resposta natural aumenta sem limites à medida que o tempo tende a infinito.
- Um sistema linear invariante no tempo é marginalmente estável caso a resposta natural não decaia nem aumente, mas permaneça constante ou oscile à medida que o tempo tende a infinito.

Outro aspecto importante no estudo da estabilidade é a análise dos polos da função de transferência do sistema. Define-se que polos no semiplano da esquerda produzem respostas naturais de decaimento exponencial puro ou senoides amortecidas. Essas respostas naturais tendem a zero à medida que o tempo tende a infinito. Assim, se os polos do sistema em malha fechada estiverem na metade esquerda do plano *s* e consequentemente tiverem parte real negativa, como representado na figura 7, o sistema será estável. Com isso, conclui-se que os

sistemas estáveis possuem funções de transferência em malha fechada com polos apenas no semiplano da esquerda (Nise, 2012, p.451).

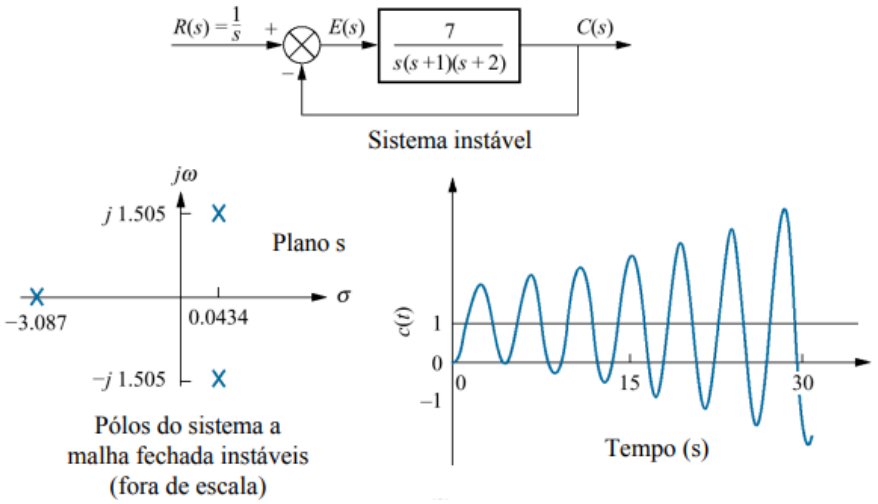
Afirma-se também que os polos no semiplano direito, visto na figura 8, produzem respostas naturais de exponenciais crescentes puras. Respostas essas que tendem ao infinito à medida que o tempo também tende ao infinito. Assim, se os polos do sistema em malha fechada estiverem na metade direita do plano s e consequentemente tiverem parte real positiva, o sistema será instável (Nise, 2012, p.451).

Figura 7: Representação e análise de um sistema estável.



Fonte: (Nise, 2012, p.453).

Figura 8: Representação e análise de um sistema instável.



Fonte: (Nise, 2012, p.453).

1.1.7 Lugar Geométrico das Raízes (LGR)

O lugar geométrico das raízes, uma representação gráfica dos polos em malha fechada à medida que um parâmetro do sistema é variado, é um método poderoso de análise e projeto para a estabilidade e resposta transitória. Os sistemas de controle com realimentação são difíceis de compreender de um ponto de vista qualitativo e, portanto, dependem fortemente da matemática (Nise, 2012, p. 580).

Então, esta representação é utilizada para descrever qualitativamente o desempenho de um sistema à medida que diversos parâmetros são alterados. O efeito da variação do ganho sobre a ultrapassagem percentual, o tempo de acomodação e o instante de pico podem ser observados detalhadamente (Nise, 2012, p.580).

Além da resposta transitória, o lugar geométrico das raízes também fornece uma representação gráfica da estabilidade do sistema. Pode-se observar claramente faixas de estabilidade, faixas de instabilidade e as condições que fazem com que um sistema entre em oscilação (Nise, 2012, p. 580). Com isso, para esboçar o lugar geométrico das raízes de um sistema qualquer, o procedimento abaixo pode ser realizado:

Tabela 1: Passo a passo do traçado do lugar geométrico das raízes.

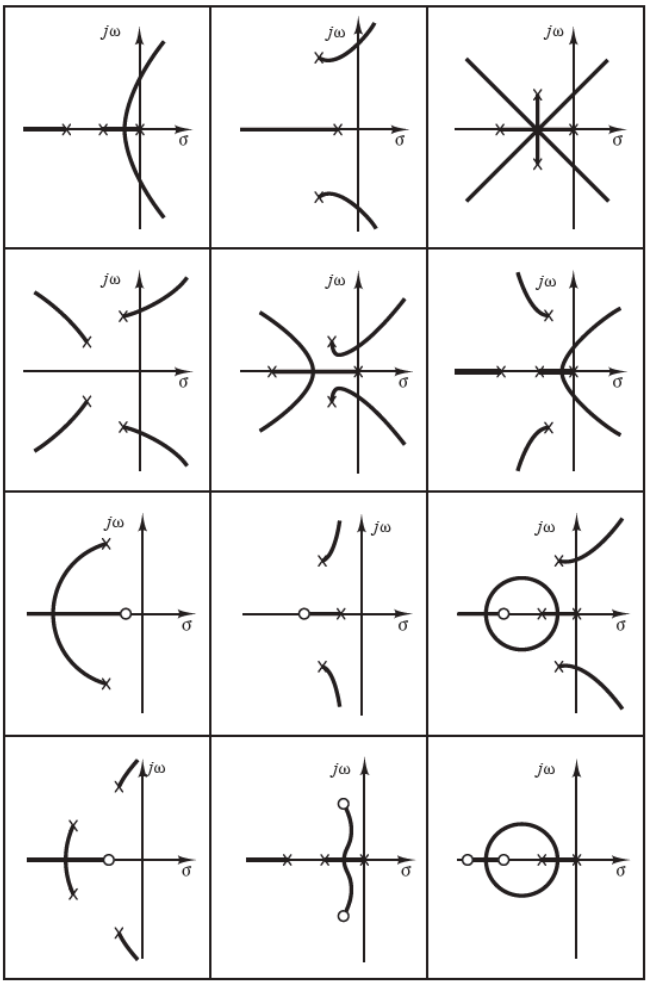
1. Localizar os polos e zeros do sistema de malha aberta	Representação: X = Polos e O = Zeros
2. Determinar as assíntotas	O número de assíntotas é a diferença entre polos e zeros do sistema: $n = n° polos - n° zeros$ Ângulo das assíntotas: $\theta = \frac{180°(2k+1)}{n° polos - n° zeros}$ para valores de k = 0,1, 2....
3. Verificar o número de ramos	Os n ramos (polos – zeros) são os traçados que partem dos polos e se dirigem aos zeros. Se o número de polos do sistema for maior que o número de zeros, alguns ramos irão para o infinito seguindo as assíntotas.
4. Localizar os pontos de partida e chegada ao eixo real	Isolar o ganho K da equação característica. Derivar a equação obtida. As raízes s_1 e s_2 da equação derivada do ganho de K, são candidatas a pontos de chegada ou partida de polos. Substituir s_1 e s_2 na equação inicial de K.

	Se o ganho for real e positivo, s_1 e s_2 pertencem ao lugar geométrico das raízes.
5. Determinar os pontos onde o lugar das raízes cruza o eixo imaginário	Os pontos são determinados através do critério de estabilidade de Routh-Hurwitz.

Fonte: (Própria, 2024).

A figura 9 exibe diversas configurações de polos e zeros e seus correspondentes lugares das raízes. O padrão do LGR depende apenas da separação relativa dos polos e zeros de malha aberta. Se o número de polos exceder o número de zeros finitos em três ou mais unidades, haverá um valor do ganho K além do qual o lugar das raízes entrará no semiplano direito do plano *s* e, assim, o sistema tornara-se instável. Com experiência no uso do método, torna-se fácil avaliar as mudanças decorrentes de modificações no número e na posição dos polos e zeros (Ogata, 2010, p.264).

Figura 9: Configurações de polos e zeros de malha aberta e os correspondentes lugares das raízes.



Fonte: (OGATA, 2010, p.264).

Uma característica importante a destacar é a simetria do LGR. Como o polinômio característico possui coeficientes reais, suas raízes só podem ser de dois tipos: raízes reais, e pares de raízes complexas conjugadas. Dessa forma, pode-se concluir que o lugar geométrico das raízes é simétrico em relação ao eixo real do plano complexo. Portanto, basta desenhar a parte acima do eixo real e, em seguida, refletir o esboço para obter a parte abaixo (E-disciplinas USP, 2024).

1.1.8 Diagrama de Bode

O método proposto por Bode, é constituído pelos gráficos de ganho e defasagem em função da frequência, representados em escala logarítmica. Uma das vantagens de se utilizar esta escala é que assim é possível representar frequências de ordens de grandeza muito diversas, adicionando-se separadamente os gráficos relativos a cada um dos termos de primeira e segunda ordem que compõe G(jω) (Pereira e Haffner, 2018, p.01).

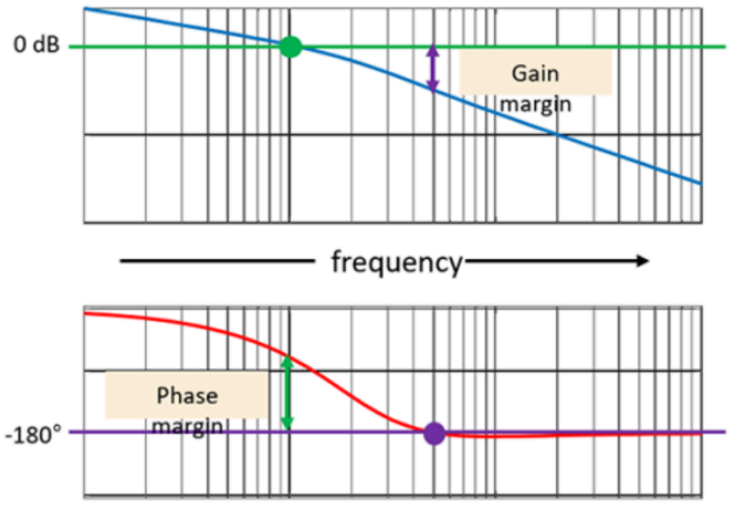
No eixo das ordenadas, os gráficos de ganho ou magnitude são expressos em decibéis, representados por $20.|G(j\omega)|$. Já os gráficos dos ângulos de fase, são representados em graus.

Os diagramas de Bode são uma alternativa para verificar a estabilidade de sistemas de controle. Essa validação é realizada por meio de duas medidas quantitativas: margem de fase e margem de ganho, que podem ser aferidas como mostrado na figura 10. Sistemas com margens de ganho e fase maiores conseguem suportar variações mais significativas em seus parâmetros antes de tornarem-se instáveis (Nise, 2012, p.841).

A margem de ganho, expressa em decibéis (dB), é obtida utilizando o diagrama de fase para determinar a frequência ω_{GM}, onde a fase é 180°. Nesta frequência observa-se o diagrama de magnitude para determinar a margem de ganho G_M, a qual é o ganho requerido para elevar a curva de magnitude até 0 dB (Nise, 2012, p.845).

A margem de fase é obtida utilizando a curva de magnitude para determinar a frequência de crossover, onde o ganho é 0 dB. Na curva de fase nesta frequência, a margem de fase ϕ_M, é a diferença entre o valor da fase e 180° (Nise, 2012, p.845).

Figura 10: Determinação das margens de ganho e fase.



Fonte: (Rohde&Schwarz – Compreender Diagramas de Bode, 2024).

1.1.9 Diagrama de Nyquist

O critério de Nyquist determina a estabilidade de um sistema de malha fechada com base na resposta em frequência do sistema de malha aberta, ou seja, ele relaciona a resposta em frequência de malha aberta G(jω) H(jω) ao número de zeros e polos de 1 + G(s)H(s) que se situam no semiplano direito do plano *s* (Ogata, 2010, p.408).

Esse critério, deduzido por H. Nyquist, é útil na engenharia de controle porque a estabilidade absoluta do sistema de malha fechada pode ser determinada graficamente a partir das curvas de resposta em frequência de malha aberta sem haver a necessidade de determinar os polos de malha fechada. Portanto, as curvas de resposta em frequência de malha aberta, obtidas analítica e experimentalmente, podem ser utilizadas na análise de estabilidade. Isso é conveniente porque, no projeto de um sistema de controle, expressões matemáticas de alguns dos componentes frequentemente não são conhecidas; apenas os dados da resposta em frequência estão disponíveis (Ogata, 2010, p.408).

De acordo com Ogata (2010), ao examinar a estabilidade de sistemas lineares de controle utilizando o critério de estabilidade de Nyquist, verifica-se que três possibilidades podem ocorrer:

1. Não existe nenhum envolvimento do ponto – 1 + j0. Isso implica que o sistema será estável se não houver polos de G(s)H(s) no semiplano direito do plano s; caso contrário, o sistema será instável.

2. Existe um ou mais envolvimentos do ponto – 1 + j no sentido anti-horário. Nesse caso, o sistema será estável se o número de envolvimentos no sentido anti-horário for o

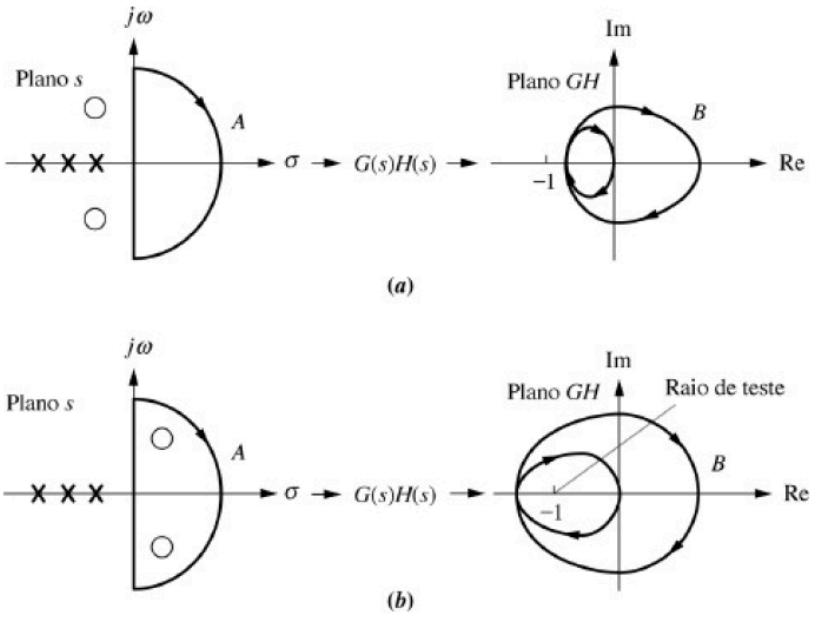
mesmo que o número de polos de G(s)H(s) no semiplano direito do plano s; caso contrário, o sistema será instável.

3. Existe um ou mais envolvimentos do ponto – 1 + j0 no sentido horário. Nesse caso, o sistema é instável.

Na figura abaixo, nota-se que o sistema é estável na parte (a), e instável na parte (b). Essa mudança no comportamento do sistema está relacionada ao valor do ganho K. Quando K assume valores baixos, o ponto -1 + j0 não é envolvido, resultando em um sistema estável. Entretanto, ao aumentar K, o lugar geométrico de G(s)H(s) passa a envolver o ponto -1 + j0, o que indica a presença de polos de malha fechada no semiplano direito do plano s. Consequentemente, o sistema torna-se instável.

Do ponto de vista da estabilidade, um valor elevado de K pode causar estabilidade deficiente ou até mesmo instabilidade, como pode ser visto nos dois casos mostrados na figura 11. Para obter uma conciliação entre precisão e estabilidade, é necessário inserir uma rede de compensação no sistema (Ogata, 2010, p.417).

Figura 11: Exemplo de contorno de Nyquist para determinar a estabilidade.

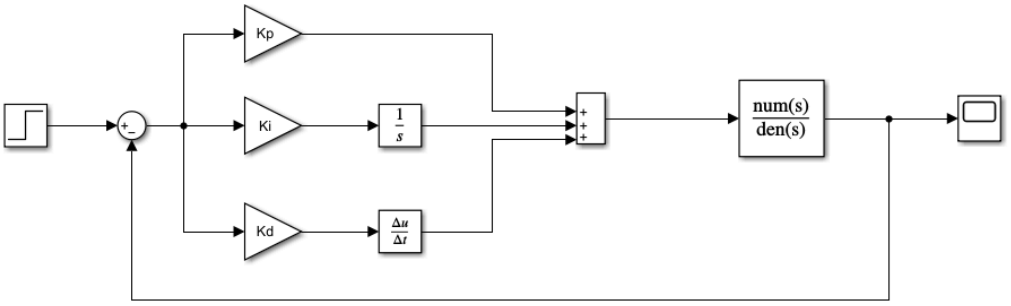


Fonte: (Nise, 2012, pág. 828).

1.1.10 Ações básicas de controle: Proporcional, Integral e Derivativo

Os três principais modos de controle em malha fechada amplamente utilizados na prática são o proporcional, integral e derivativo. Na figura 12, observa-se o esquema geral da implementação das três ações em conjunto, onde cada uma tem seus benefícios para que a variável controlada do sistema não apresente erros de regime permanente, tenha uma redução no tempo de subida e overshoot. Desta forma, o controlador PID é amplamente utilizado devido à sua eficácia em alcançar uma resposta rápida e precisa.

Figura 12: Diagrama de blocos de um Controlador PID.



Fonte: (Própria, 2024).

A seguir, os controladores: proporcional (P), proporcional integral (PI), proporcional derivativo (PD), proporcional integral derivativo (PID) são abordados separadamente, suas características e efeitos sobre um sistema de controle são especificados, e suas respostas típicas são ilustradas na figura 13.

1. *Controlador Proporcional (P)*

A relação entre a saída e a entrada do controlador é conhecida como ganho proporcional (Kp), representado através da equação 10. Quanto maior for esse ganho, menor será o erro no estado estacionário. No entanto, o aumento do ganho também pode prolongar o tempo de acomodação e, em alguns casos, tornar o sistema mais oscilatório e potencialmente instável. Esse tipo de controlador não introduz novos polos e zeros no sistema principal, apenas ajusta o ganho original (Araújo, 2007).

Equação 10: Representação do Controlador Proporcional.

$$U(s) = K_p . E(s)$$

Onde U(s) = Saída do controlador, K_P = Ganho Proporcional e E(s) = sinal de erro atuante.

2. *Controlador Proporcional Integral (PI)*

A ação integral refere-se à taxa de variação do sinal de saída em relação à entrada. Esta ação melhora a resposta em regime permanente ao eliminar o erro estacionário. No entanto, pode impactar negativamente o regime transitório ao adicionar polos no sistema, o que pode aumentar o tempo de acomodação e potencialmente desestabilizá-lo (Araújo, 2007, p.26).

O controlador PI, representado na equação 11, combina a ação proporcional com a ação integral. Assim, a resposta transitória é aprimorada pela ação proporcional, enquanto a ação integral corrige o erro em estado estacionário (Araújo, 2007).

Equação 11: Representação do Controlador Proporcional Integral.

$$U(s) = \left(K_P + \frac{K_I}{s}\right).\ E(s)$$

Sendo U(s) = Saída do controlador, K_P = Ganho Proporcional, K_I = Ganho Integral e E(s) = sinal de erro atuante.

3. Controlador Proporcional Derivativo (PD)

Embora um controlador puramente derivativo não possa ser implementado na prática, a inclusão da ação derivativa junto com a ação proporcional adiciona um zero ao sistema, o que melhora o desempenho no regime transitório e aumenta a estabilidade relativa, além de diminuir o tempo de acomodação. No entanto, essa abordagem tem a desvantagem de aumentar o tempo de subida e, por não influenciar no regime permanente, não corrige o erro de estado estacionário (ARAÚJO, 2007, p.27). Este tipo de controlador é representado através da equação abaixo:

Equação 12: Representação do Controlador Proporcional Derivativo.

$$U(s) = \left(K_P + K_D\,.\ s\right)E(s)$$

As variáveis significam: U(s) = Saída do controlador, K_P = Ganho Proporcional, K_D = Ganho Derivativo e E(s) = sinal de erro atuante.

4. Controlador Proporcional Integral Derivativo (PID)

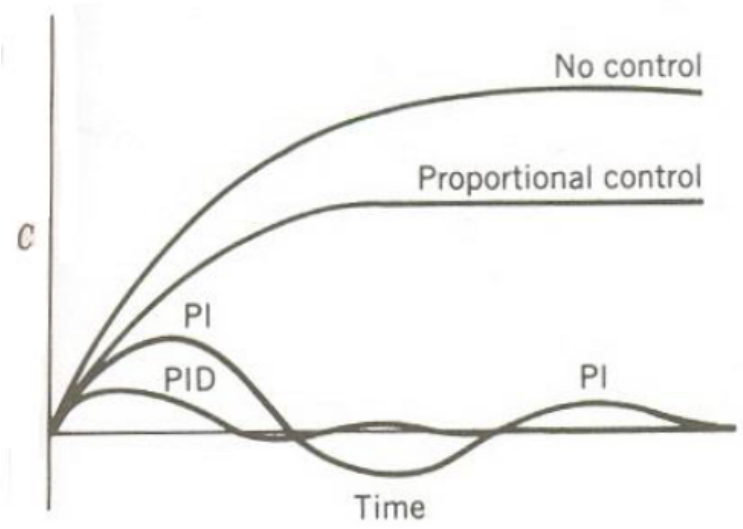
O PID une as ações proporcional, integral e derivativa num só controlador, além de atuar tanto no regime transitório quanto no regime permanente. Este controlador, representado através da equação 13, é capaz de proporcionar uma resposta rápida e estável, minimizar o erro de estado estacionário, e reduzir o tempo de subida. Por estas características, é amplamente utilizado em diversas áreas, como controle de temperatura, velocidade, posição, pressão etc.

Equação 13: Representação do Controlador PID.

$$U(s) = \left(K_P + \frac{K_I}{s} + K_D \,.\, s\right)E(s)$$

Sendo U(s) = Saída do controlador, K_P = Ganho Proporcional, K_D = Ganho Derivativo, K_I = Ganho Integral e E(s) = sinal de erro atuante.

Figura 13: Respostas típicas de controladores.



Fonte: (E-disciplinas USP, 2024).

1.2 Recursos computacionais para a implementação da Biblioteca Virtual

1.2.1 Software Matlab

O Matlab é um software de alto desempenho amplamente utilizado para a computação numérica, análise de dados, visualização, simulação de sistemas etc. Esse programa oferece um ambiente interativo e flexível no qual os usuários podem executar cálculos complexos e realizar análises de maneira eficiente. Sendo bastante utilizado em diversas áreas, como engenharia, física, matemática, ciências da computação, economia e outros campos científicos e tecnológicos.

Especificamente neste trabalho, o Matlab será utilizado para a área de sistemas de controle, visto que é possível realizar todos os estágios de desenvolvimento dos processos determinados. Desde a modelagem e simulação da resposta transitória da planta, análise da estabilidade, representação dos diagramas de Bode e Nyquist, até a implementação dos

controladores Proporcional (P), Proporcional Integral (PI), Proporcional Derivativo (PD), Proporcional Integral Derivativo (PID).

O fato de oferecer esses benefícios, faz com que a criação e implementação da biblioteca virtual neste software seja prática e efetiva. Também é importante ressaltar que, o Matlab é disponibilizado gratuitamente para os estudantes da Universidade do Estado do Amazonas (UEA). Portanto, optou-se por esta ferramenta para realizar o projeto da plataforma de sistemas de controle.

1.2.2 App Designer: Definição

A biblioteca virtual será implementada no App Designer, ferramenta integrada no ambiente de desenvolvimento Matlab, projetada para criar aplicativos de interface gráfica de usuário (GUI). O App Designer oferece uma abordagem interativa e visual, onde os usuários podem inserir diversos elementos gráficos como botões, tabelas e controles deslizantes (*sliders*) em uma tela de design (*Design View*).

Abaixo, os principais recursos e funcionalidades do App Designer são descritos:

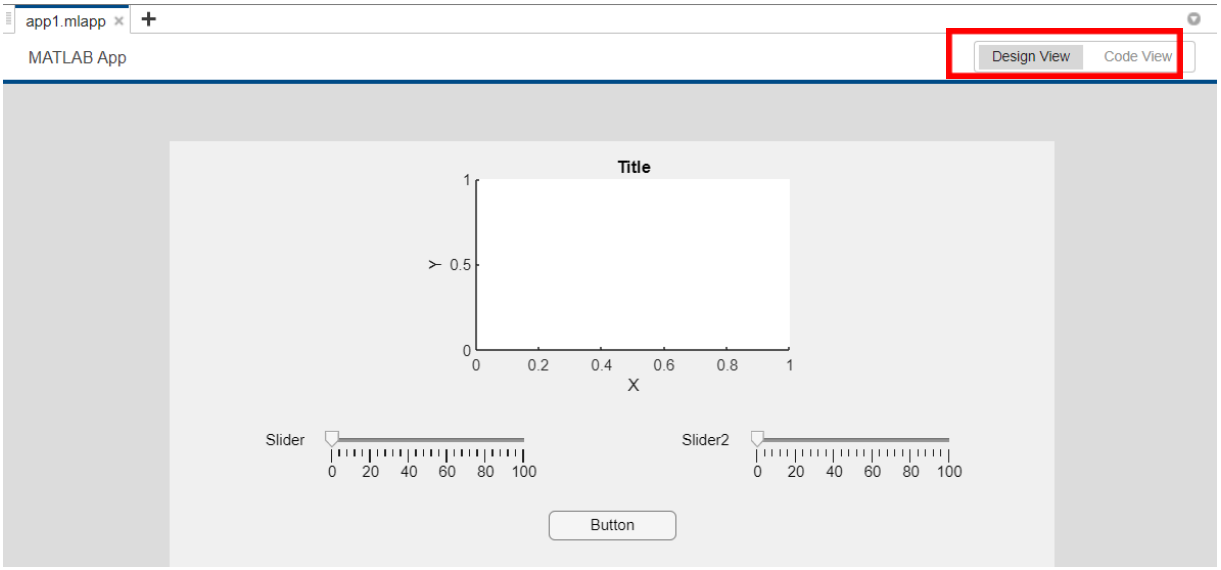
- Componentes Gráficos: Ampla variedade de componentes pré-construídos, como botões, caixas de texto, menus, painéis, controles deslizantes, tabelas e gráficos.
- Personalização e Estilo: Oferece opções para personalizar a aparência e o estilo dos componentes gráficos para atender a necessidades específicas.
- Código Matlab Integrado: Permite a adição de *callbacks* e funções auxiliares, para definir o comportamento dos componentes gráficos adicionados.
- Ferramentas de Depuração: Inclui funcionalidades de depuração para testar e solucionar problemas no aplicativo em desenvolvimento. O usuário pode inserir breakpoints em partes do código para compreender o seu funcionamento.
- Compartilhamento e Distribuição: Permite a criação de aplicativos autônomos que podem ser compartilhados com outros usuários, através da compactação em arquivos do instalador diretamente da barra de ferramentas do Matlab, ou da criação de aplicativos desktop ou web independente.

1.2.3 Estrutura do App Designer

O App Designer, conforme mostrado na Figura 14, é dividido em duas seções principais para o desenvolvimento de aplicativos: *Design View* e *Code View*. Cada seção desempenha um papel específico no processo de criação e programação das interfaces gráficas.

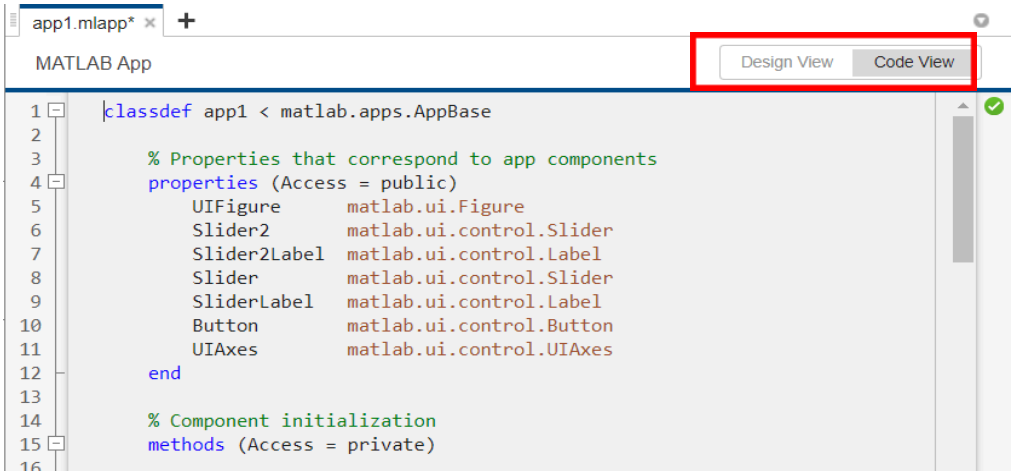
O *Design View* organiza visualmente a interface do usuário, ao permitir a inserção de componentes, e ajustar suas propriedades como fonte, tamanho, cor etc. No *Code View*, o usuário escreve o código que define o comportamento desses componentes. As duas seções operam em conjunto, proporcionando uma experiência de desenvolvimento integrada e completa.

Figura 14: Divisão do App Designer: Seção *Design View*.



Fonte: (Própria, 2024).

Figura 15: Divisão do App Designer: Seção *Code View*.



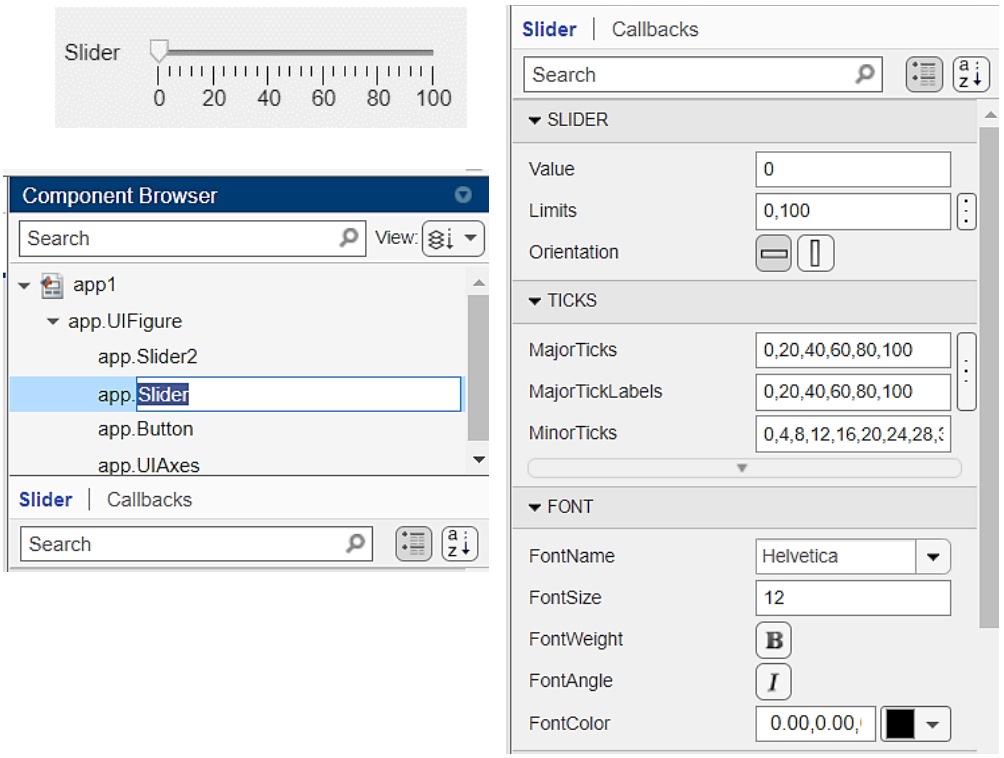
Fonte: (Própria, 2024).

1.2.4 *Design View*: Características e propriedades

Esta seção do App Designer fornece um conjunto rico de ferramentas de layout, com uma extensa biblioteca com mais de 55 componentes UI, para projetar aplicativos interativos, modernos e com aparência profissional. Quaisquer alterações feitas no *Design View* relacionadas a cor, tamanho, fonte, visibilidade e posição na interface gráfica, são automaticamente refletidas no *Code View*. Dessa maneira, o usuário pode configurar aspectos gráficos do seu aplicativo sem escrever códigos (Mathworks, 2024).

Ao adicionar um elemento, o mesmo aparece na *Component Browser*, lista hierárquica de todos os componentes presentes no aplicativo, onde é possível renomear e modificar as suas propriedades. No exemplo mostrado na figura 16, para o componente controle deslizante (*slider*), é possível alterar o seu limite, que por padrão vem definido de 0 a 100. Assim como também, pode-se modificar a sua disposição na interface, seja horizontalmente ou verticalmente. Outra mudança importante, é a alteração dos pontos destacados (*ticks*), como pode ser visto na figura 10, neste exemplo os valores são 20, 40, 60, 80, 100. Além de, definir o tipo e tamanho da fonte. É importante destacar que, cada componente tem suas propriedades específicas para alteração.

Figura 16: Menu *Component Browser* do controle deslizante (*slider*).



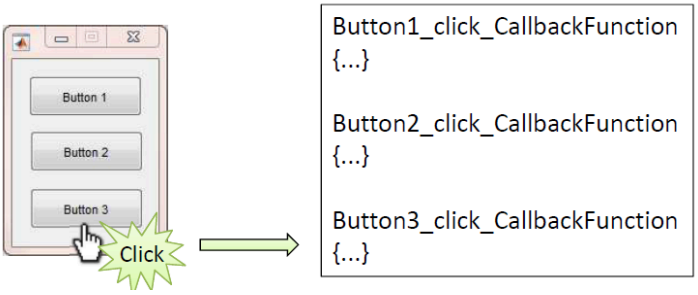
Fonte: (Própria, 2024).

1.2.5 *Code View*: Características e propriedades

O *Code View* é a seção onde o usuário escreve e edita o código que define o comportamento dos componentes da interface gráfica. Esta ferramenta permite integrar a lógica de programação diretamente com os elementos visuais inseridos no *Design View*. O fluxo do código é guiado por indicações externas, chamadas de eventos. Estes são descritos abaixo por Lopes, Rangel e Marta (2023), e o seu comportamento é mostrado na figura 17.

- Eventos são as diferentes ações que usuários podem realizar sobre os componentes adicionados à interface.
- Cada evento está associado à uma função chamada *callback*, disparada quando verifica-se a ocorrência de tal evento, que define a reação do programa.
- No App Designer, os *callbacks* são associadas a métodos, no contexto de programação orientada a objetos, a uma classe que descreve a aplicação gráfica.

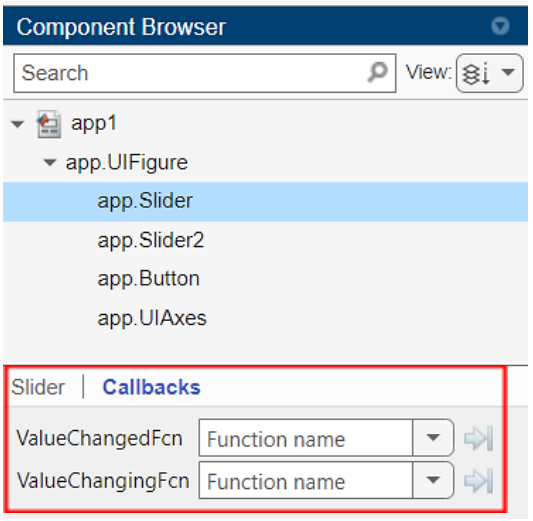
Figura 17: Resposta a eventos nos componentes gráficos.



Fonte: (Lopes; Rangel; Martha, 2023).

A maioria dos componentes, a exceção das *labels* e lâmpadas, possuem pelo menos um *callback*, vinculado a uma interação específica com o elemento. Para consultar a lista de *callbacks* que um componente suporta, selecione o elemento e clique na guia "*Callbacks"* disponível no *Component Browser*. Na figura 18, por exemplo, observa-se que para o componente controle deslizante (*slider*), dois tipos de callbacks podem ser criados*:* *ValueChangedFcn* e *ValueChangingFcn* (Mathworks, 2024).

Figura 18: *Callbacks* disponíveis no componente *slider*.



Fonte: (Própria, 2024).

Ao criar um *callback* para o componente, o App Designer gera a função no *Code View* e insere o seu cursor no local onde o código deve ser escrito. Os *callbacks* possuem dois tipos de argumentos de entrada:

- *App* - O *app.object* é utilizado para acessar componentes UI do aplicativo, bem como outras variáveis armazenadas como propriedades. O app argumento fornece o app objeto para o seu callback. É possível acessar qualquer componente, e todas as suas particularidades, dentro de qualquer *callback* utilizando esta sintaxe (Mathworks, 2024). Por exemplo, o comando abaixo define a propriedade valor (*Value*) do controle deslizante de ganho (*GanhoSlider*) para 10.

```
app.GanhoSlider.Value= 10;
```

- *Event* - Objeto que contém informações específicas sobre a interação do usuário com o componente da interface, ou seja, sobre o evento que acionou o callback. O *event* argumento fornece um objeto com propriedades diferentes, com informações importantes, para o tipo de interação à qual o *callback* responde. Por exemplo, o *event* argumento para o *ValueChangingFcn callback* de um controle deslizante (*slider*) contém uma propriedade chamada *Value*. Essa propriedade armazena o valor do slider conforme o usuário o move, antes de soltar o mouse. Abaixo, encontra-se o exemplo de um callback do controle deslizante que usa o *event* argumento para fazer o *Tempo Edit Field(Numeric)* rastrear o valor do slider (Mathworks, 2024).

```
%Value changing function: TempoSlider
function TempoSliderValueChanging(app, event)
        app.Tempo.Value = event.Value;
end
```

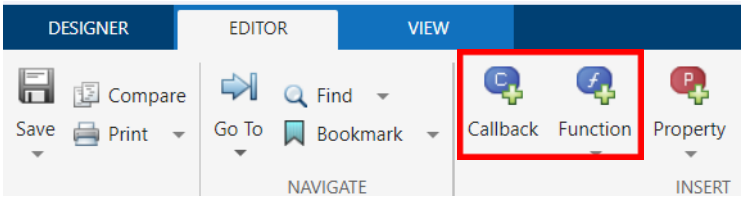
Outra parte importante do *Code View* são as funções, classificadas em dois tipos: funções auxiliares (*Helper Functions*) e funções de inicialização (*Startup Functions*). As *Helper Functions* são funções definidas no aplicativo, chamadas em diferentes partes do código. Por exemplo, para atualizar um gráfico após o usuário alterar um número no campo de edição ou selecionar um item em uma lista suspensa, a criação de uma *Helper Function* permite que o aplicativo tenha uma única fonte de comandos comuns e evite manter o código redundante (Mathworks, 2024).

Existem dois tipos de *Helper Functions*: Funções Privadas (*Private Functions*), que só podem ser chamadas dentro do seu aplicativo, e Funções Públicas (*Public Functions*), que podem podem ser chamadas dentro ou fora do seu aplicativo. Funções privadas são comumente usadas em aplicativos de janela única, enquanto funções públicas são comumente usadas em aplicativos de múltiplas janelas (Mathworks, 2024).

O App Designer também permite a criação de uma função especial, executada quando o aplicativo é iniciado, mas antes que o usuário interaja com a UI. Essa função é chamada de *Startup Function*, sendo útil para definir valores padrões, inicializar variáveis ou executar comandos que afetam o estado inicial do aplicativo. Por exemplo, pode-se utilizar a *Startup Function* para exibir um gráfico padrão ou mostrar a lista de valores padrões de uma tabela (Mathworks, 2024).

As *Helper Function*, *Startup Function* e *Callbacks* podem ser criados na Aba "Editor" do *Code View,* ilustrado abaixo na figura 19.

Figura 19: Criação de *callbacks*, *functions* e *properties* no Editor do *Code View*.



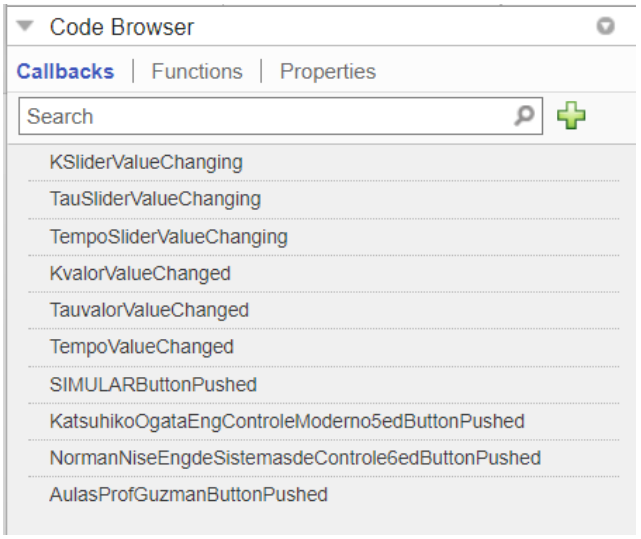
Fonte: (Própria, 2024).

Também vale destacar, o menu *Code Browser* do *Code View*, mostrado na figura 20, que disponibiliza as seguintes funcionalidades para a manipulação desses três elementos:

- Adicionar, excluir ou renomear qualquer *callback*, *function* ou *property* personalizadas no app.
- Clicar em um item na guia *callbacks* ou *functions* direciona o editor para a seção correspondente do seu código.

- Reorganizar a ordem dos *callbacks*: basta selecionar o que o usuário deseja mover e, em seguida, arrastá-lo e soltá-lo em sua nova posição na lista. Este recurso também reposiciona o callback no editor.
- Localizar rapidamente um *callback*, *function* ou *property* ao digitar parte de seu nome na barra de pesquisa.

Figura 20: Menu *Code Browser* do *Code View*.



Fonte: (Própria, 2024).

2. METODOLOGIA

O trabalho em questão é uma pesquisa aplicada, e tem como objetivo a realização de uma pesquisa exploratória e explicativa relacionada ao referencial teórico, e logo após é feita a aplicação direta dos conceitos definidos. Os procedimentos técnicos de pesquisa bibliográfica e experimental são utilizados. Assim como também, o método de abordagem hipotético-dedutivo e o método de procedimento monográfico foram empregados para realizar a elaboração.

Pesquisas bibliográficas sobre metodologias ativas de ensino, e como os laboratórios e bibliotecas virtuais podem ser utilizados para melhorar o processo de aprendizagem dos alunos foram realizadas. Em seguida, foi feita a revisão abrangente da literatura no âmbito de sistemas de controle, com a definição, importância de suas aplicações, os principais componentes do processo, a definição de função de transferência, como verificar a

estabilidade de sistemas, o lugar geométrico das raízes, diagrama de Bode, diagrama de Nyquist e ações básicas de controle.

Além do mais, o referencial teórico inclui a descrição do software utilizado para a implementação do projeto, o Matlab. Assim como também, aborda os tópicos que definem e caracterizam a ferramenta utilizada para desenvolver as interfaces gráficas, o App Designer.

Após o estudo e revisão bibliográfica de sistemas de controle, foram definidos quais assuntos seriam abordados e como as interfaces seriam divididas. Dessa maneira, optou-se por fazer sete interfaces gráficas, nomeadas como: Resposta Temporal de Sistemas de 1° ordem, Resposta Dinâmica de Sistemas de 2° ordem, Estabilidade em Sistemas de Controle, Lugar Geométrico das Raízes, Diagrama de Bode, Diagrama de Nyquist e Ações Básicas de Controle.

Os sistemas escolhidos foram selecionados com base nas análises que poderiam ser realizadas sobre eles, e nas condições que poderiam ser exploradas para estudar os conceitos de sistemas de controle. Os processos selecionados foram: o controle do nível de líquido, sistema massa mola amortecedor, circuito elétrico com amplificador operacional, e três funções de transferência disponíveis como exemplos no livro Ogata.

Posterior a delimitação, as interfaces, arquivos com extensão *.mlapp*, foram desenvolvidas de maneira simples e básica, para analisar o comportamento dos sistemas e garantir que as saídas e análises de estabilidade estivessem de acordo com o esperado. Após confirmar o funcionamento básico dos processos, a estrutura e o layout padrão dos scripts foram definidos no *Design View*.

Nessa etapa, os componentes gráficos, como botões, controles deslizantes (*sliders*) e campos de edição (*Edit Fields*), foram adicionados e configurados. Também nesta parte, dois elementos essenciais foram utilizados: o *Grid Layout* e o *Tab Group*. Para criar um aplicativo redimensionável, onde os elementos se ajustam ao tamanho da tela, foi empregado o *Grid Layout*. Já o *Tab Group*, foi utilizado para organizar os tópicos e permitir a inserção de mais conteúdos nas interfaces.

Por fim, após o desenvolvimento do aplicativo, realização de testes e validações das simulações e scripts, o projeto foi compilado para compartilhamento da biblioteca virtual de sistemas de controle. A fim de avaliar a receptividade e primeira impressão dos estudantes, foi

conduzida uma breve pesquisa de aceitação, após a apresentação da plataforma durante as aulas de Sistemas de Controle I do curso de Engenharia Elétrica.

Figura 21: Etapas para implementação da Biblioteca Virtual.



Fonte: (Própria, 2024).

3. DESENVOLVIMENTO DA BIBLIOTECA VIRTUAL

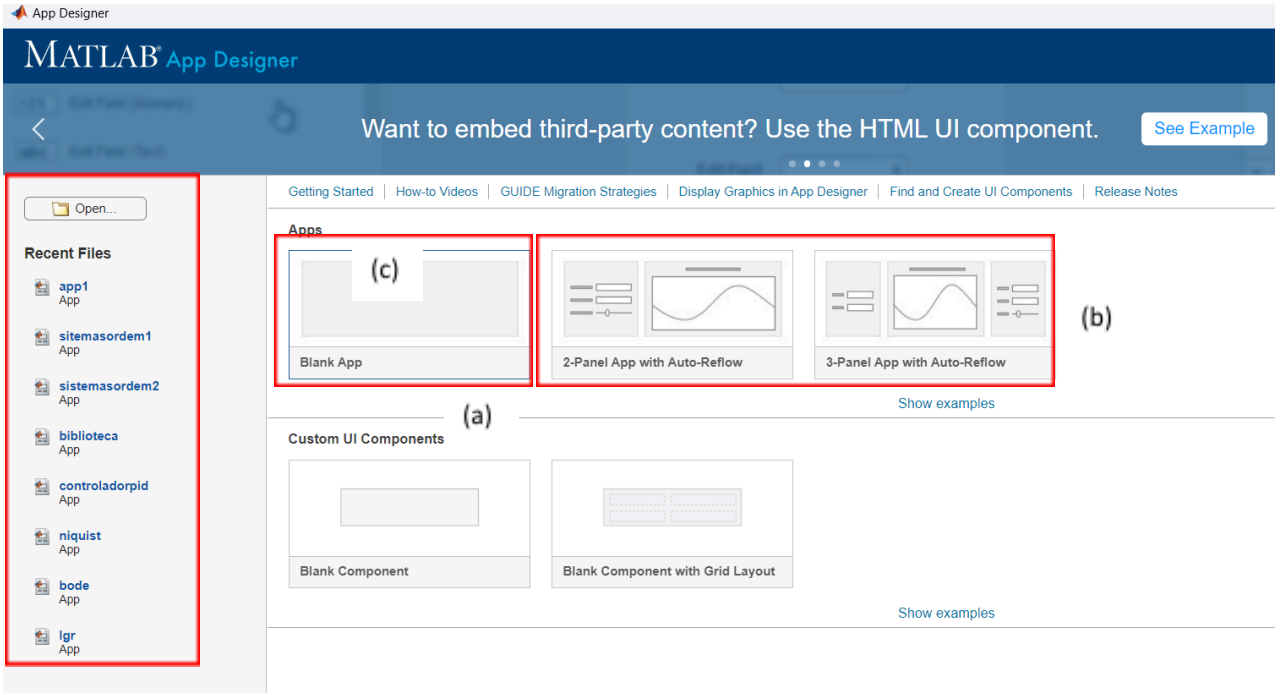
Nesta parte do trabalho, são abordados os processos para implementação do projeto, responsáveis pela compilação de um aplicativo funcional e interativo. Destaca-se os comandos e componentes utilizados, a definição do layout, o resultado das interfaces gráficas e a compilação da plataforma.

3.1 Comandos e componentes gráficos utilizados no App Designer

Para iniciar a implementação do sistema, deve-se acessar o App Designer. Para isso, o usuário pode ir a tela inicial do Matlab, localizar a aba "*Apps*", e clicar no botão "*Design*

app". Ou também, acessar através da "*Command Window*", ao digitar "*appdesigner*" e pressionar enter. Com isso, a tela da figura 22 será exibida.

Figura 22: Página inicial do App Designer.



Fonte: (PRÓPRIA, 2024).

Na página acima, o usuário tem três opções para iniciar: Abrir um *Blank App* (a) e construir seu aplicativo do zero sem nenhuma pré-configuração definida; acessar uma interface pronta oferecida pela Mathworks (b) com um tutorial de primeiros passos; ou também abrir suas próprias interfaces (c) em desenvolvimento ao clicar em Open.

Para este trabalho, diante de um *Blank App*, os seguintes componentes gráficos e comandos foram utilizados no desenvolvimento da Biblioteca Virtual:

Tabela 2: Lista de componentes gráficos utilizados na implementação da Biblioteca Virtual.

Componente Gráfico	Ícone	Descrição
Label	A Label	Inserir texto estático, utilizado para adicionar descrições, instruções e informações.
Text Area	Text Area	Inserir e editar várias linhas de texto. Com funcionalidades a mais quando comparado ao *Label*, como rolagem (*scrollable*), para navegar pelo conteúdo inserido, por exemplo.

Edit Field (Text)	 Edit Field (Text)	O usuário pode inserir e editar textos, é usado para coletar entradas textuais para o programa.
Edit Field (Numeric)	 Edit Field (Numeric)	Permite ao usuário inserir e modificar valores numéricos, sendo útil quando é necessário obter entradas de dados precisas.
Image	 Image	Exibir imagens, como um ícone ou logotipo, por exemplo.
Axes	 Axes	Exibir gráficos, essencial para representar dados de forma visual e interativa. Com propriedades de *zoom in*, *zoom out*, restaurar escala original, mover-se ao longo do gráfico, salvar e copiar a imagem.
Radio Button Group	 Radio Button Group	Agrupa uma série de botões de opção, que permite ao usuário selecionar apenas uma alternativa entre várias disponíveis.
Slider	 Slider	Selecionar o valor numérico contínuo ou discreto dentro de um intervalo definido. Pode ser usado para ajustar parâmetros e controlar a dinâmica de gráficos.
Button	 Button	Responde a uma função ou evento quando pressionado.
Grid Layout	 Grid Layout	Posicionar os componentes UI ao longo de linhas e colunas de uma grade invisível.
Tab Group	 Tab Group	Agrupar e gerenciar guias.

Fonte: (Mathworks, 2024).

Tabela 3: Comandos utilizados na implementação da Biblioteca Virtual.

Comando	Função
tf(G);	Criar modelos de função de transferência de valor real ou de valor complexo. Converter modelos de sistemas dinâmicos para o formato de função de transferência.tf.

rlocus(G);	Plotar o gráfico de lugar geométrico das raízes de sistemas de controle.
bode(G);	Plotar o Diagrama de Bode da resposta em frequência com dados de magnitude e fase.
nyquist(G);	Plotar Diagrama de Nyquist da resposta em frequência.
feedback (G, 1);	Calcular o sistema de malha fechada a partir de um sistema de malha aberta especificado.
step(G);	Plotar a resposta ao degrau de um sistema dinâmico.
lsim (G, u, t);	Plotar a resposta de sistemas dinâmicos para entradas arbitrárias. Sendo u = entrada e t = tempo de amostragem.
tfdata (G);	Acessar dados da função de transferência.
zpkdata (G);	Acessar os dados de polos, zeros e ganhos de uma função de transferência.
findobj (object, property, value);	Encontrar objetos gráficos com propriedades específicas.
copyobj (object);	Copiar objetos gráficos e seus descendentes.
squeeze(magnitude);	Remover dimensões unitárias de uma matriz ou array multidimensional. Pode ser usado para manipular e ajustar dados que possuem dimensões adicionais não necessárias para visualização ou análise.
str2num (numerador);	Converter matriz de caracteres ou string em matriz numérica.
poly2str (numerador);	Retornar um polinômio como string.
plot (step(G));	Plotar gráficos bidimensionais.
title (‘Resposta ao degrau’);	Inserir título em gráficos.
semilogx (app.Axes);	Plotar gráficos em escalas logarítmicas.
cla (app.Axes);	Limpar o gráfico, remover as informações de uma iteração anterior.
hold (app.PosAxes, 'on');	Manter o gráfico atual para adicionar novos gráficos. Ativa o modo de retenção, onde novas plotagens são adicionadas ao gráfico existente sem limpar os dados ou a formatação previamente plotada.
axis auto;	Atualizar gráficos a cada nova iteração.
web(link);	Abrir página na web ou arquivo no navegador.
winopen (arquivo);	Abrir arquivos ou URLs com aplicativos padrão do sistema operacional. Útil para abrir documentos, planilhas e arquivos de texto.

run("sitemasordem1.mlapp");	Utilizado para executar uma interface desenvolvida. O Matlab compila e executa o código do aplicativo, abre a interface gráfica e permite que o usário interaja com todos os componentes e funcionalidades configurados.

Fonte: (Mathworks, 2024).

3.2 Layout do aplicativo

A estrutura das telas do aplicativo de sistemas de controle foi fundamental para assegurar uma experiência de usuário intuitiva. Na configuração do layout, os componentes *Tab Group* e *Grid Layout*, foram essenciais na delimitação sistemática dos assuntos e distribuição dos elementos pela interface.

O conteúdo dos laboratórios foi dividido com o *Tab Group*, que organiza a interface em várias abas, e facilita a navegação e categorização de informações e elementos dentro do aplicativo. Esse recurso é adequado para aplicações com muitos componentes gráficos a serem exibidos, pois mantém a interface limpa e organizada (Mathworks, 2024).

Uma característica importante do *Tab Group* é o fato de cada aba poder conter seus próprios componentes, como botões, tabelas, gráficos e *sliders*. Esses elementos são independentes e podem até repetir-se em outras guias sem causar conflitos de *callbacks* no *Code View*. Outro aspecto importante, são as propriedades ajustáveis das guias, como nome, ordem e visibilidade.

No projeto, as interfaces foram divididas em três tipos de abas:

1. Simulação: A primeira guia é dedicada às simulações e análises dos sistemas de controle. Nela, foram inseridos os principais componentes gráficos, como *axes*, *sliders* e os *radio button group*, que estão relacionados às *Helper Functions* e à maioria dos *callbacks*.
2. Exemplos: A segunda guia apresenta um breve resumo do assunto e sugere exercícios de sistemas pré-selecionados com o intuito de que o usuário simule, observe o comportamento e, assim, compreenda os conceitos de sistemas de controle relacionados aquele respectivo assunto. Nestas guias, foram utilizados apenas dois componentes gráficos: *Labels* e *images*.
3. Bibliografia: Última guia, onde são disponibilizados os livros Ogata e Nise, devidamente divididos de acordo com o tema da respectiva interface, além do link do drive de videoaulas do professor Guzman, com as explicações dos

assuntos. O objetivo desta última aba, é incentivar os alunos a aprofundar-se mais sobre os conteúdos, e encontrar outros exercícios para simulação.

Com o objetivo de construir um aplicativo redimensionável, que ajusta-se ao tamanho da tela, foi utilizado o componente *Grid Layout*. Este recurso organiza e alinha os componentes em uma grade flexível e estruturada dentro da interface gráfica do usuário. Esta propriedade, facilita a criação de layouts responsivos, o que garante componentes ajustáveis ao redimensionamento da janela do aplicativo. O uso do Grid Layout mostrou-se mais vantajoso em comparação ao redimensionamento automático (*Resize Children*) e aos *callbacks* de .*SizeChangedFcn* (Mathworks, 2024).

O *Grid Layout* abrange toda a janela do aplicativo ou contêiner em que for inserido, e permanece invisível, a menos que o usuário o esteja configurando ativamente. É importante ressaltar, a configuração das linhas (*RowHeight*) e colunas (*ColumnWidth*), no qual foi utilizada a opção *Weighted*, com a finalidade de ajustar proporcionalmente os componentes, baseado no espaço disponível na interface. Por último, também foram definidos os espaçamentos entre as linhas (*RowSpacing*) e colunas (*ColumnSpacing*), de acordo com os valores mais adequados para cada tela do aplicativo.

Em resumo, a combinação do *Tab Group* e do *Grid Layout* no App Designer proporcionou uma estrutura organizada e responsiva para as interfaces gráficas de sistemas de controle, e melhorou significativamente a praticidade, interatividade e eficiência do aplicativo.

3.3 Descrição das Interfaces Gráficas

Esta seção descreve o objetivo, o sistema sugerido para simulação e o funcionamento de cada interface gráfica. Além de, mostrar detalhadamente os componentes gráficos, *callbacks* e funções utilizados no seu desenvolvimento.

- ***INTERFACE 1: Resposta Temporal de Sistemas de 1° ordem***

Objetivo: Realizar simulações para analisar como sistemas de primeira ordem respondem a entradas do tipo degrau e rampa. E assim, compreender a função das variáveis do sistema, e como estas afetam o seu comportamento.

Com o intuito de mostrar ao usuário as variações do ganho K e da constante de tempo (τ), selecionou-se o modelo do controle do nível de líquido em um tanque, onde a saída é regulada por uma válvula. O sistema é modelado através da função de transferência ilustrada na equação 14, onde a variável de saída é a altura do nível de líquido *h* do tanque, e a variável de entrada é a vazão volumétrica *qi* do reservatório:

Equação 14: Função de transferência do controle do nível de líquido.

$$G(s) = \frac{R}{RC.s+1}$$

Este exemplo está localizado na segunda guia. No entanto, os usuários não estão restritos a esse sistema, é possível simular qualquer modelo representado no formato da equação geral de sistemas de 1° ordem (Equação 2).

Funcionamento: O usuário seleciona os valores de K, τ e tempo de simulação através dos *sliders* ou *Edit Field (Numeric)*. Depois, escolhe a qual entrada o sistema será submetido, degrau ou rampa. Após isso, clicar no botão “SIMULAR”, para verificar o gráfico de resposta.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 4: Recursos utilizados na implementação da Interface 1.

Componentes Gráficos	Função
3 *Sliders*	Controlar o intervalo de 0 a 20 do ganho K, de 0 a 20 da constante de tempo (τ), e de 0 a 5000 do tempo de simulação.
3 *Edit Field (Numeric)*	Mostrar e inserir valores no intervalo dos *sliders* de ganho, constante de tempo e tempo de simulação.
1 *Radio Button Group*	Selecionar a entrada degrau ou rampa.

1 *Button*	O evento associado a ele, plota o gráfico de acordo com as especificações definidas nos *sliders*.
1 *Axes*	Exibe o gráfico da resposta ao degrau e rampa de sistemas de 1° ordem.
Callbacks	**Tipo**
K *Slider* τ *Slider* Tempo de simulação *Slider*	*ValueChangingFcn*: O valor é modificado no *Edit Field (Numeric)* durante a interação.
K *Edit Field* τ *Edit Field* Tempo de simulação *Edit Field*	*ValueChangedFcn*: Atualiza o *slider* quando um valor é modificado
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *respostasistema(app)* é executada
Função	**Objetivo**
respostasistema(app)	Simular a resposta com os valores inseridos nos três *sliders*. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 2: Resposta Dinâmica de Sistemas de 2° ordem***

Objetivo: Realizar simulações para observar como sistemas de segunda ordem respondem a entradas do tipo degrau e rampa. E assim, analisar os diferentes tipos de respostas: não amortecida, subamortecida, superamortecida, criticamente amortecida.

Através do sistema massa mola amortecedor, proposto na guia de “Exemplos”, é possível verificar como a variação do coeficiente de amortecimento pode afetar o comportamento do sistema. Nesta situação, constatou-se três tipos de respostas: subamortecida, superamortecida e criticamente amortecida.

O sistema massa-mola-amortecedor é descrito pela função de transferência abaixo, que relaciona a força aplicada versus o deslocamento. Este e outros sistemas de segunda ordem, representados pela equação geral (Equação 8), podem ser simulados na plataforma.

Equação 15: Função de transferência do sistema massa mola amortecedor.

$$G(s) = \frac{1}{m.s^2+b.s+K}$$

Funcionamento: O usuário seleciona os valores da frequência natural (ω_n), coeficiente de amortecimento (ξ) e tempo de simulação através dos *sliders* ou *Edit Field (Numeric)*. Em

seguida, escolhe a qual entrada o sistema será submetido, degrau ou rampa. Por fim, clica no botão “SIMULAR”, para verificar o gráfico de resposta.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 5: Recursos utilizados na implementação da Interface 2.

Componentes Gráficos	**Função**
3 *Sliders*	Controlar o intervalo de 0 a 20 da frequência natural (ω_n), de 0 a 20 do coeficiente de amortecimento (ζ), e de 0 a 5000 do tempo de simulação.
3 *Edit Field (Numeric)*	Mostrar e inserir valores no intervalo dos *sliders* de frequência natural, coeficiente de amortecimento e tempo de simulação.
1 *Radio Button Group*	Selecionar a entrada degrau ou rampa.
1 *Button*	O evento associado a ele, plota o gráfico de acordo com as especificações definidas nos *sliders*.
1 *Axes*	Exibe o gráfico da resposta ao degrau e rampa de sistemas de 2° ordem.
Callbacks	**Tipo**
ω_n *Slider* ζ *Slider* Tempo de simulação *Slider*	*ValueChangingFcn*: O valor é modificado no *Edit Field (Numeric)* durante a interação.
ω_n *Edit Field* ζ *Edit Field* Tempo de simulação *Edit Field*	*ValueChangedFcn*: Atualiza o *slider* quando um valor é modificado.
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *respostasistema2(app)* é executada.
Função	**Objetivo**
Respostasistema2(app)	Simular a resposta do sistema com os valores inseridos nos três *sliders*. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 3: Estabilidade em Sistemas de Controle***

Objetivo: Determinar a estabilidade dos sistemas ao analisar a posição dos polos no plano geométrico, e ao observar a resposta a entrada degrau.

O circuito elétrico com amplificador operacional é representado através da equação de transferência abaixo, que relaciona a tensão de entrada versus a tensão de saída.

Equação 16: Função de transferência do do circuito elétrico com amplificador operacional.

$$\frac{E_o(s)}{E_i(s)} = \frac{1}{\left(R_1.C_1.R_2.C_2\right)s^2+\left[R_2.C_2+R_1.C_2+\left(\frac{R_1}{R_3}\right).R_2.C_2\right]s+\left(\frac{R_1}{R_3}\right)}$$

Por meio deste modelo, ao variar a realimentação, mostra-se o movimento dos polos do semiplano esquerdo para o semiplano direito, ou seja, do sistema entrando na região de instabilidade. Assim como também, nota-se a mudança da resposta ao degrau do sistema, onde inicialmente, convergia para um valor constante, mas que posteriormente, apresentou oscilações persistentes e crescentes.

Funcionamento: Insira os coeficientes do numerador e denominador da função de transferência do sistema nos respectivos *Edit Field*s. Em seguida, clique no botão "SIMULAR" para observar os gráficos.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 6: Recursos utilizados na implementação da Interface 3.

Componentes Gráficos	**Função**
2 *Edit Field (Text)*	Inserir os coeficientes do numerador e denominador da função de transferência.
Label	Mostrar a função de transferência em formato latex.
1 *Button*	O evento associado a ele, plota os gráficos da função de transferência especificada.
2 *Axes*	Um gráfico plota a posição dos polos e zeros no plano geométrico, e o outro, exibe a resposta ao degrau.
Callbacks	**Tipo**
Numerador *Edit Field* Denominador *Edit Field*	*ValueChangedFcn*: Atualiza os coeficientes da função de transferência
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *resposta(app)* é executada
Função	**Objetivo**
resposta(app)	Simular os gráficos de posição e resposta baseado nos coeficientes inseridos. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 4: Lugar Geométrico das Raízes***

Objetivo: Traçar o gráfico do lugar das raízes para examinar sistemas de controle. Observar o movimento dos polos do sistema de malha fechada no plano complexo à medida que o parâmetro de ganho é alterado, o que ajudará na avaliação da estabilidade e desempenho do sistema.

Nas abas “Exemplo” encontra-se o passo a passo para esboçar o lugar geométrico das raízes do sistema especificado pela equação 17.

Equação 17: Sistema para simulação do LGR.

$$G(s) = \frac{K(s+2)}{s^2+2s+3} \qquad H(s) = 1$$

Funcionamento: Insira os coeficientes do numerador e denominador da função de transferência do sistema nos respectivos *Edit Fields*. Em seguida, clique no botão “SIMULAR” para observar o gráfico do LGR.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 7: Recursos utilizados na implementação da Interface 4.

Componentes Gráficos	**Função**
2 *Edit Field (Text)*	Inserir os coeficientes do numerador e denominador da função de transferência.
Label	Mostrar a função de transferência em formato latex.
1 *Button*	O evento associado a ele, plota o gráfico do LGR da função de transferência especificada.
1 *Axes*	Exibe o gráfico do lugar geométrico das raízes.
Callbacks	**Tipo**
Numerador *Edit Field* Denominador *Edit Field*	*ValueChangedFcn*: Atualiza os coeficientes da função de transferência.
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *lugargeometrico(app)* é executada.
Função	**Objetivo**
lugargeometrico(app)	Simular o gráfico do LGR baseado nos coeficientes inseridos. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 5: Diagrama de Bode***

Objetivo: Analisar a resposta em frequência de sistemas através dos gráficos de magnitude e fase.

Com o intuito de analisar o diagrama de Bode, nas abas “Exemplo”, sugere-se simular o sistema descrito pela equação 18:

Equação 18: Sistema para simulação do Diagrama de Bode.

$$G(s) = \frac{25}{s^2+4s+25}$$

Desta maneira, pode-se verificar o ganho, magnitude e frequência natural. Além de, calcular a margem de ganho e a margem de fase.

Funcionamento: Insira os coeficientes do numerador e denominador da função de transferência do sistema nos respectivos *Edit Fields*. Em seguida, clique no botão “SIMULAR” para observar o Diagrama de Bode.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 8: Recursos utilizados na implementação da Interface 5.

Componentes Gráficos	**Função**
2 *Edit Field (Text)*	Inserir os coeficientes do numerador e denominador da função de transferência.
Label	Mostrar a função de transferência em formato latex.
1 *Button*	O evento associado a ele, plota o Diagrama de Bode da função de transferência especificada.
1 *Axes*	Exibe o Diagrama de Bode do sistema.
Callbacks	**Tipo**
Numerador *Edit Field* Denominador *Edit Field*	*ValueChangedFcn*: Atualiza os coeficientes da função de transferência.
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *Bode(app)* é executada.
Função	**Objetivo**
Bode(app)	Simular o Diagrama de Bode baseado nos coeficientes inseridos. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 6: Diagrama de Nyquist***

Objetivo: Simular o Diagrama de Nyquist, que ilustra a resposta em frequência do sistema no plano complexo. Dessa forma, é possível calcular a margem de ganho e determinar a estabilidade de sistemas em malha fechada.

O sistema representado pela equação 19, é sugerido para a simulação do Diagrama de Nyquist, e análise da estabilidade através da equação Z = N + P.

Equação 19: Representação do sistema para simulação do Diagrama de Nyquist.

$$G(s) = \frac{K}{(s+0{,}5)(s+1)(s+1{,}5)}$$

Primeiramente, o diagrama é avaliado para o ganho K = 2, e observa-se que o sistema é estável. Em seguida, simula-se o sistema com o ganho K = 10, constatando que nesta situação, o sistema é instável. Uma sugestão excelente é utilizar a interface 3 para simular o sistema de malha fechada com os respectivos ganhos, e confirmar a estabilidade e a instabilidade do sistema conforme descrito.

Funcionamento: Insira os coeficientes do numerador e denominador da função de transferência do sistema nos respectivos *Edit Fields*. Em seguida, clique no botão "SIMULAR" para observar o Diagrama de Nyquist.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 9: Recursos utilizados na implementação da Interface 6.

Componentes Gráficos	**Função**
2 *Edit Field (Text)*	Inserir os coeficientes do numerador e denominador da função de transferência.
Label	Mostrar a função de transferência em formato latex.
1 *Button*	O evento associado a ele, plota o Diagrama de Nyquist da função de transferência especificada.
1 *Axes*	Exibe o Diagrama de Nyquist do sistema.
Callbacks	**Tipo**
Numerador *Edit Field* Denominador *Edit Field*	*ValueChangedFcn*: Atualiza os coeficientes da função de transferência.

Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *nyquist(app)* é executada.
Função	**Objetivo**
nyquist(app)	Simular o Diagrama de Nyquist baseado nos coeficientes inseridos. Código disponível no anexo A.

Fonte: (Própria, 2024).

- ***INTERFACE 7: Ações básicas de controle: Proporcional, Integral e Derivativo***

Objetivo: Estudar as respostas e os impactos das ações de controle proporcional, integral e derivativa sobre o tempo de subida, tempo de acomodação, overshoot, erro de regime permanente e erro de estado estacionário. Ao final, implementar essas três ações de controle de forma conjunta.

Novamente, recomenda-se implementar separadamente os controladores proporcional (P), proporcional derivativo (PD) e proporcional integral (PI) no sistema massa mola amortecedor. Em seguida, implementar o controlador proporcional integral derivativo (PID), e constatar a redução do erro de estado estacionário, o aumento do tempo de subida e a redução do overshoot.

Funcionamento: Insira os coeficientes do numerador e denominador da função de transferência do sistema nos respectivos *Edit Fields*. Em seguida, digite os valores dos ganhos proporcional (Kp), integral (Ki) e derivativo (Kd). Por último, clique no botão “SIMULAR” para observar a resposta do sistema.

Abaixo, segue a tabela dos principais componentes gráficos, *callbacks* e funções utilizadas para plotar os gráficos e implementar as funcionalidades da interface gráfica.

Tabela 10: Recursos utilizados na implementação da Interface 7.

Componentes Gráficos	Função
2 *Edit Field (Text)*	Inserir os coeficientes do numerador e denominador da função de transferência.
Label	Mostrar a função de transferência em formato latex.
3 *Edit Field (Numeric)*	Digitar os valores dos ganhos proporcional (Kp), integral (Ki) e derivativo (Kd)
1 *Radio Button Group*	Selecionar o tipo de controlador: P, PI, PD ou PID

1 *Button*	O evento associado a ele, plota a resposta do sistema com o devido controlador aplicado.
1 *Axes*	Exibe a resposta do sistema.
Callbacks	**Tipo**
Numerador *Edit Field* Denominador *Edit Field*	*ValueChangedFcn*: Atualiza os coeficientes da função de transferência.
Kp *Edit Field(Numeric)* Ki *Edit Field(Numeric)* Kd *Edit Field(Numeric)*	*ValueChangedFcn*: Atualiza os valores do ganho proporcional, integral e derivativo
Simular *Button*	*ButtonPushedFcn*: Quando o botão é pressionado, a função *controlador(app)* é executada.
Função	**Objetivo**
controlador(app)	Simular a resposta do sistema com os respectivos valores de ganho, e o tipo de controlador aplicado. Código disponível no anexo A.

Fonte: (Própria, 2024).

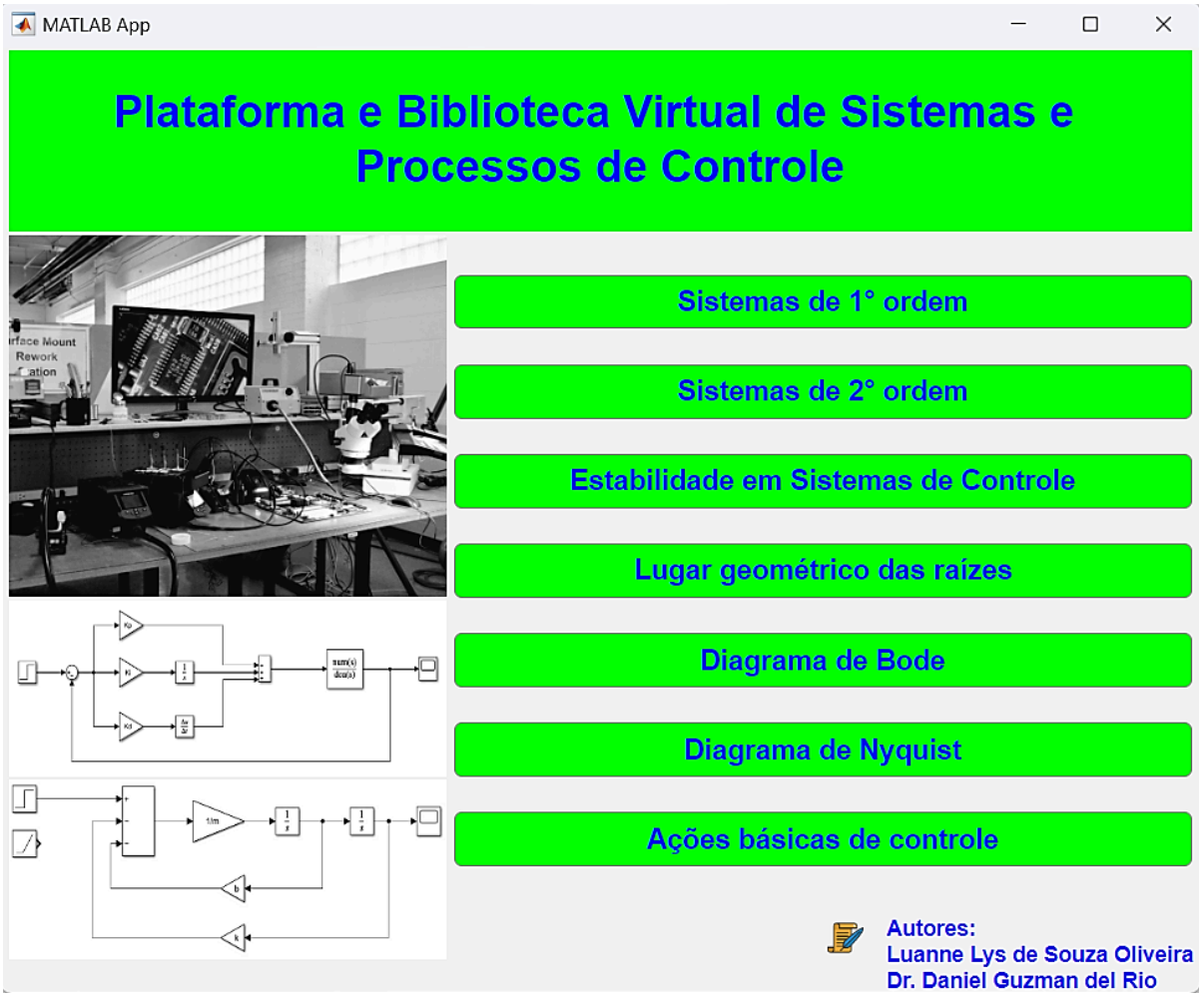
3.4 Apresentação das Interfaces Gráficas

Após a descrição das telas do aplicativo, abaixo encontra-se a versão final da biblioteca virtual.

- ***Capa da Biblioteca Virtual***

Esta é a página inicial da Plataforma e Biblioteca Virtual de Sistemas e Processos de Controle. A partir dela, através do *callback run* associado aos botões, o usuário pode abrir as interfaces gráficas disponíveis. E assim, simular e analisar o sistema que desejar.

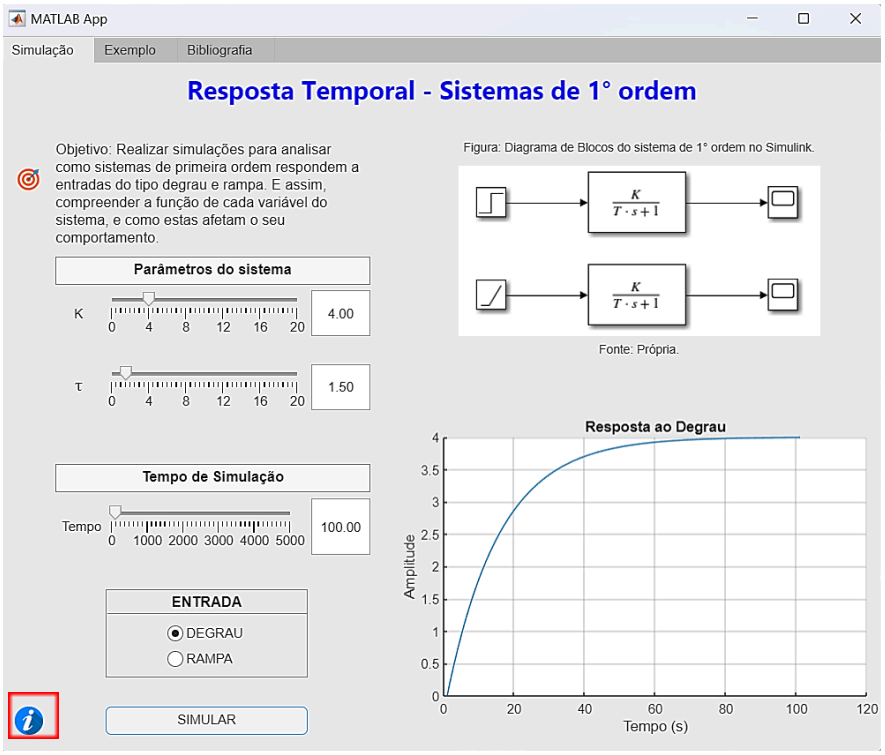
Figura 23: Página inicial da Biblioteca Virtual.



Fonte: (Própria, 2024).

- ***INTERFACE 1: Resposta Temporal de Sistemas de 1° ordem***

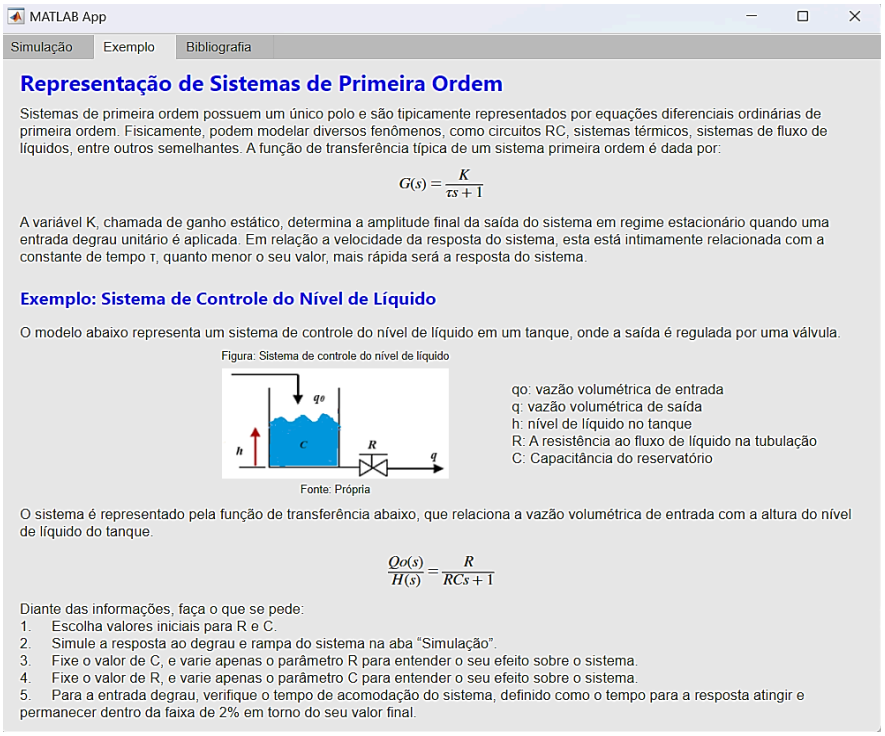
Figura 24: Guia “Simulação” da Interface Gráfica 1.



Fonte: (Própria, 2024).

O ícone destacado, no canto inferior esquerdo, contém dicas para a simulação dos sistemas. E está presente em todas as guias de Simulação, os textos encontram-se disponíveis no anexo B.

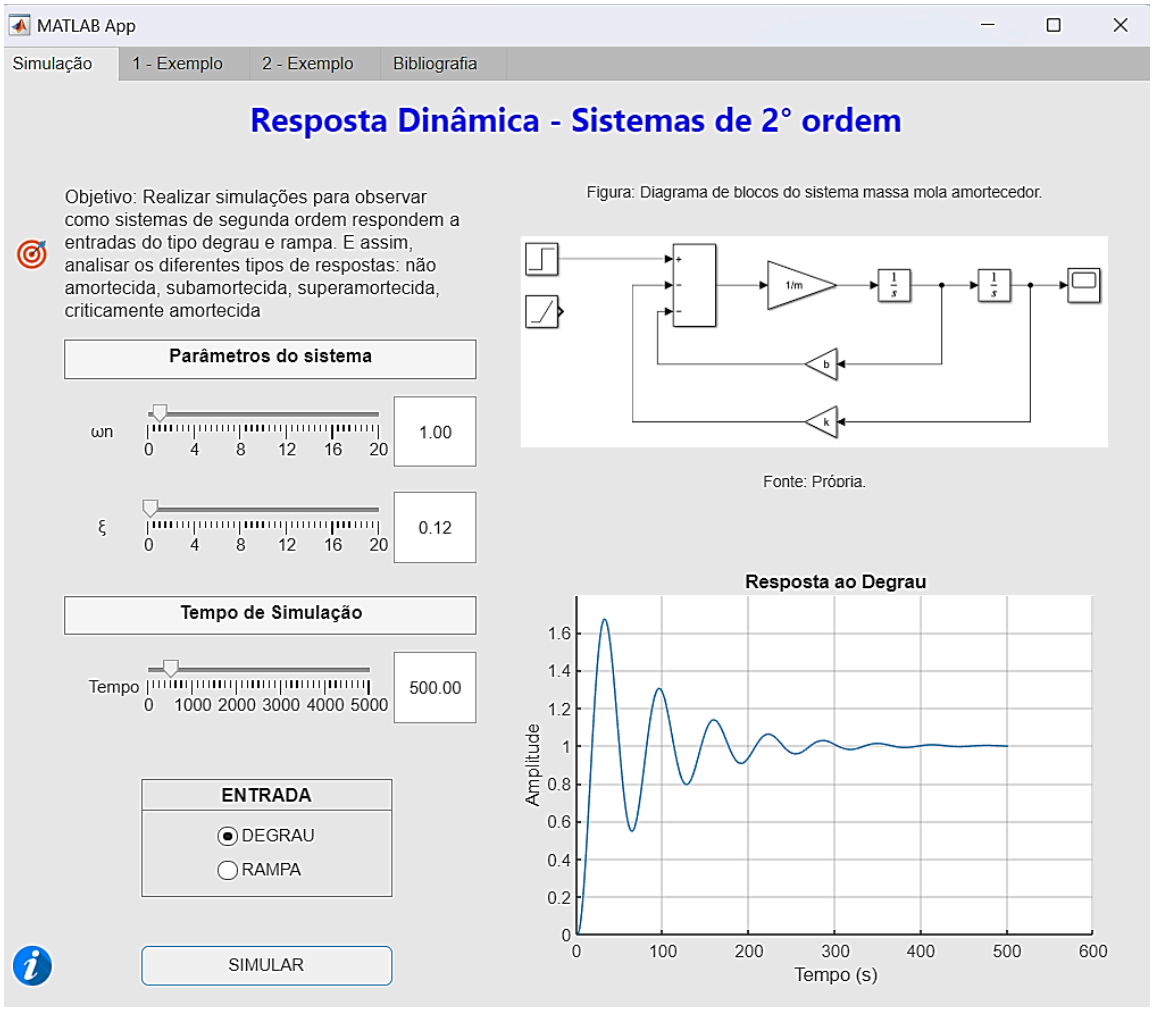
Figura 25: Guia “Exemplo” da Interface Gráfica 1.



Fonte: (Própria, 2024).

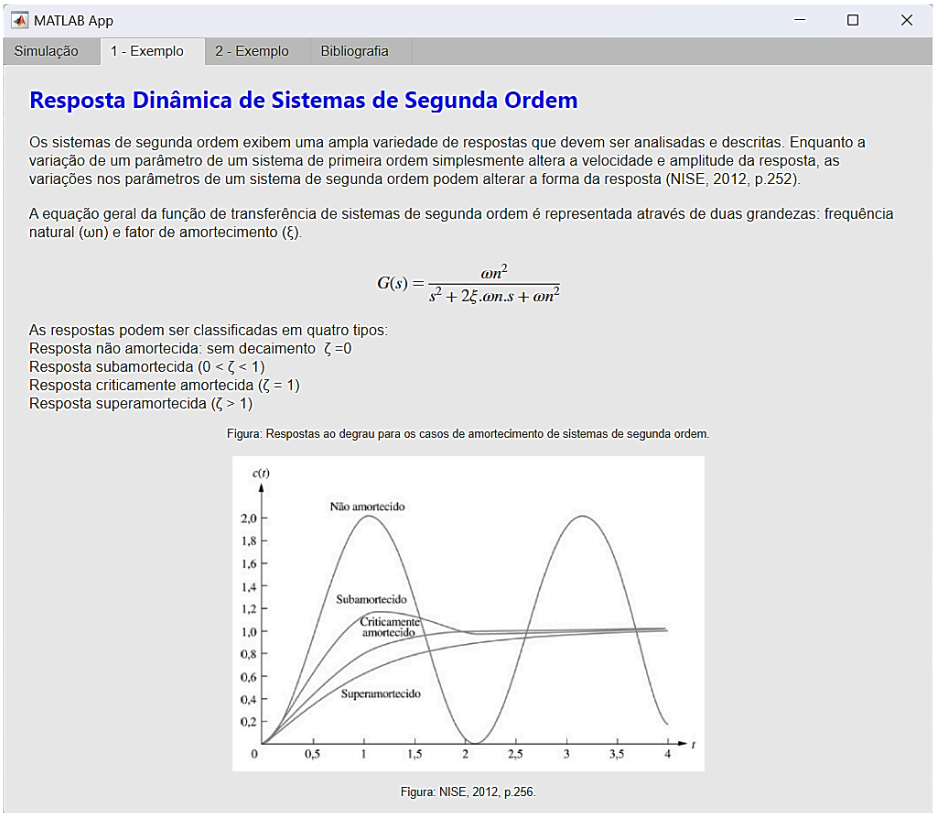
- ***INTERFACE 2: Resposta Dinâmica de Sistemas de 2° ordem***

Figura 26: Guia “Simulação” da Interface Gráfica 2.



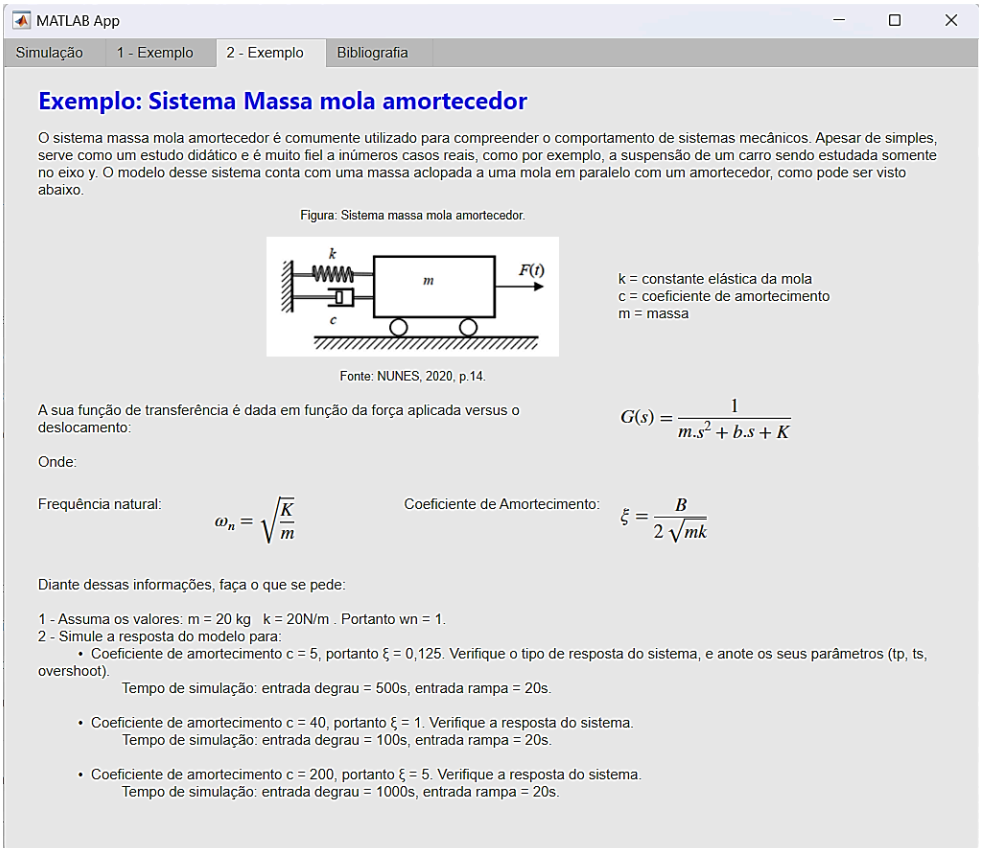
Fonte: (Própria, 2024).

Figura 27: Guia “1 - Exemplo” da Interface Gráfica 2.



Fonte: (Própria, 2024).

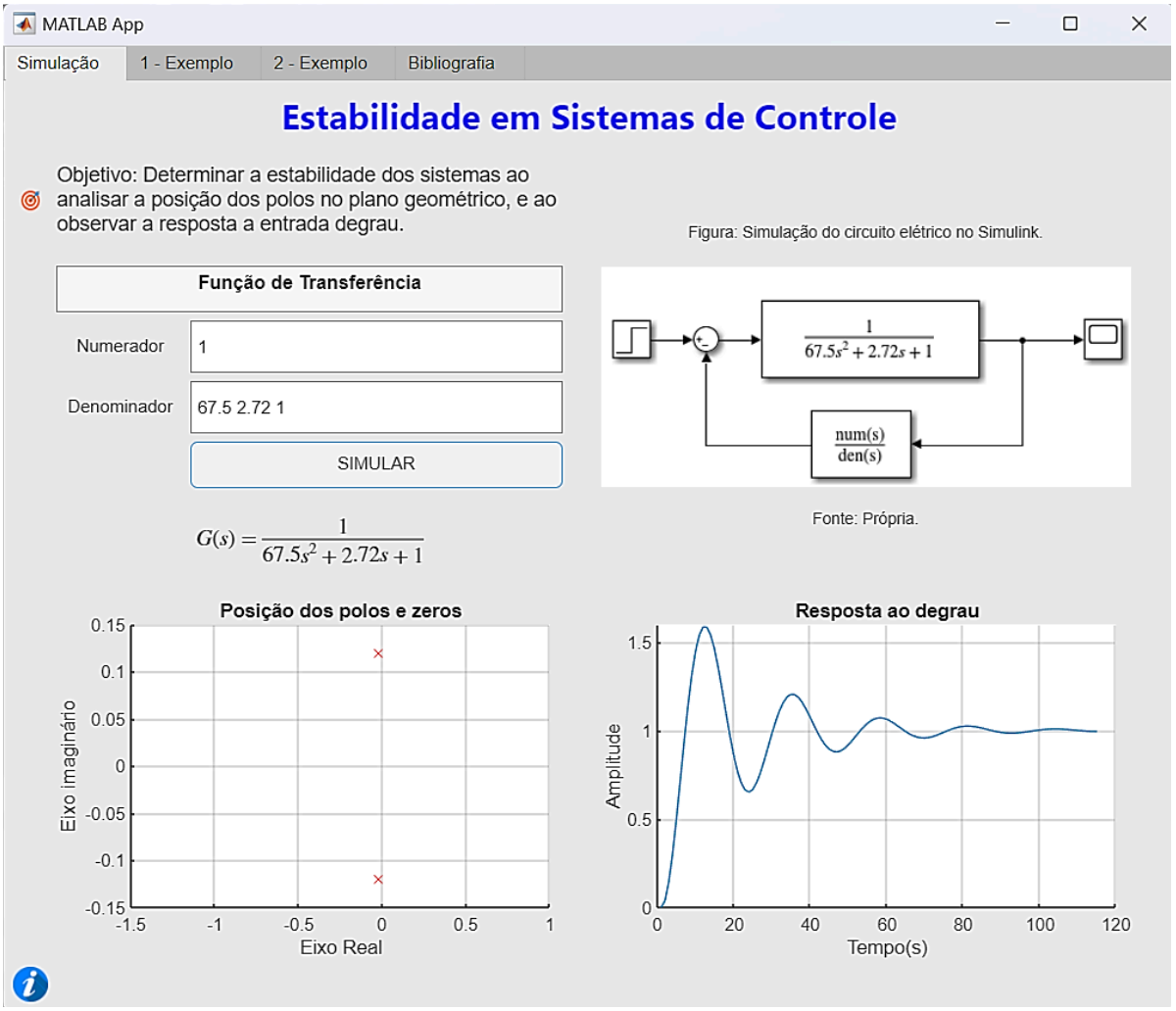
Figura 28: Guia “2 - Exemplo” da Interface Gráfica 2.



Fonte: (Própria, 2024).

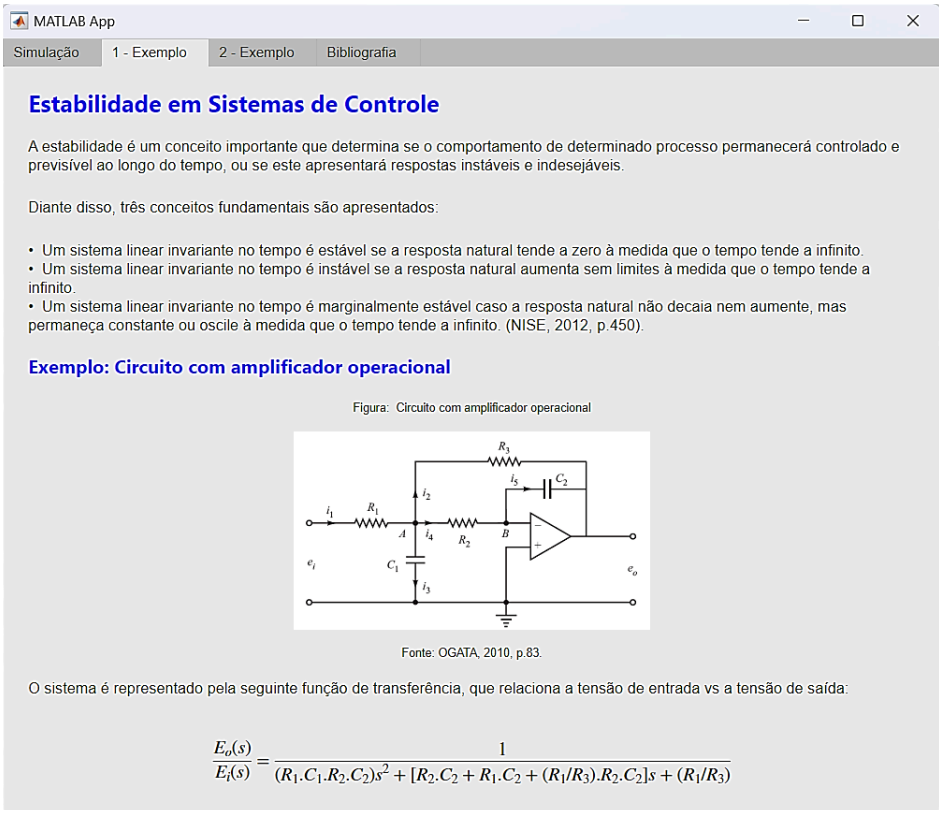
- ***INTERFACE 3: Estabilidade em Sistemas de Controle***

Figura 29: Guia “Simulação” da Interface Gráfica 3.



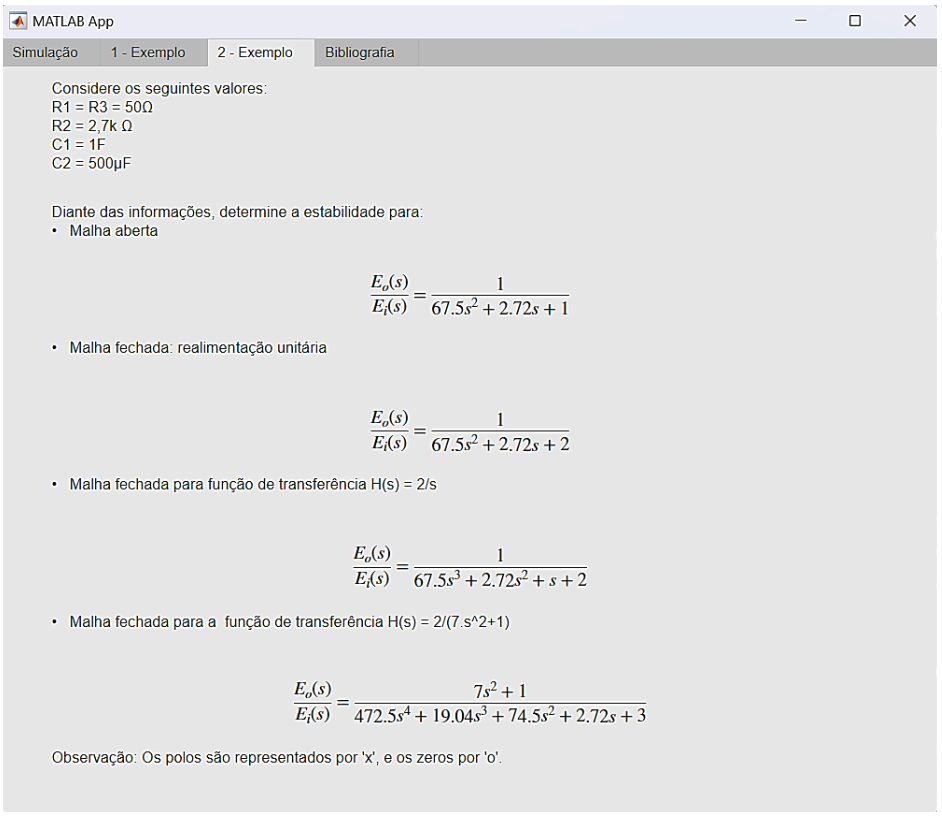
Fonte: (Própria, 2024).

Figura 30: Guia “1 - Exemplo” da Interface Gráfica 3.



Fonte: (Própria, 2024).

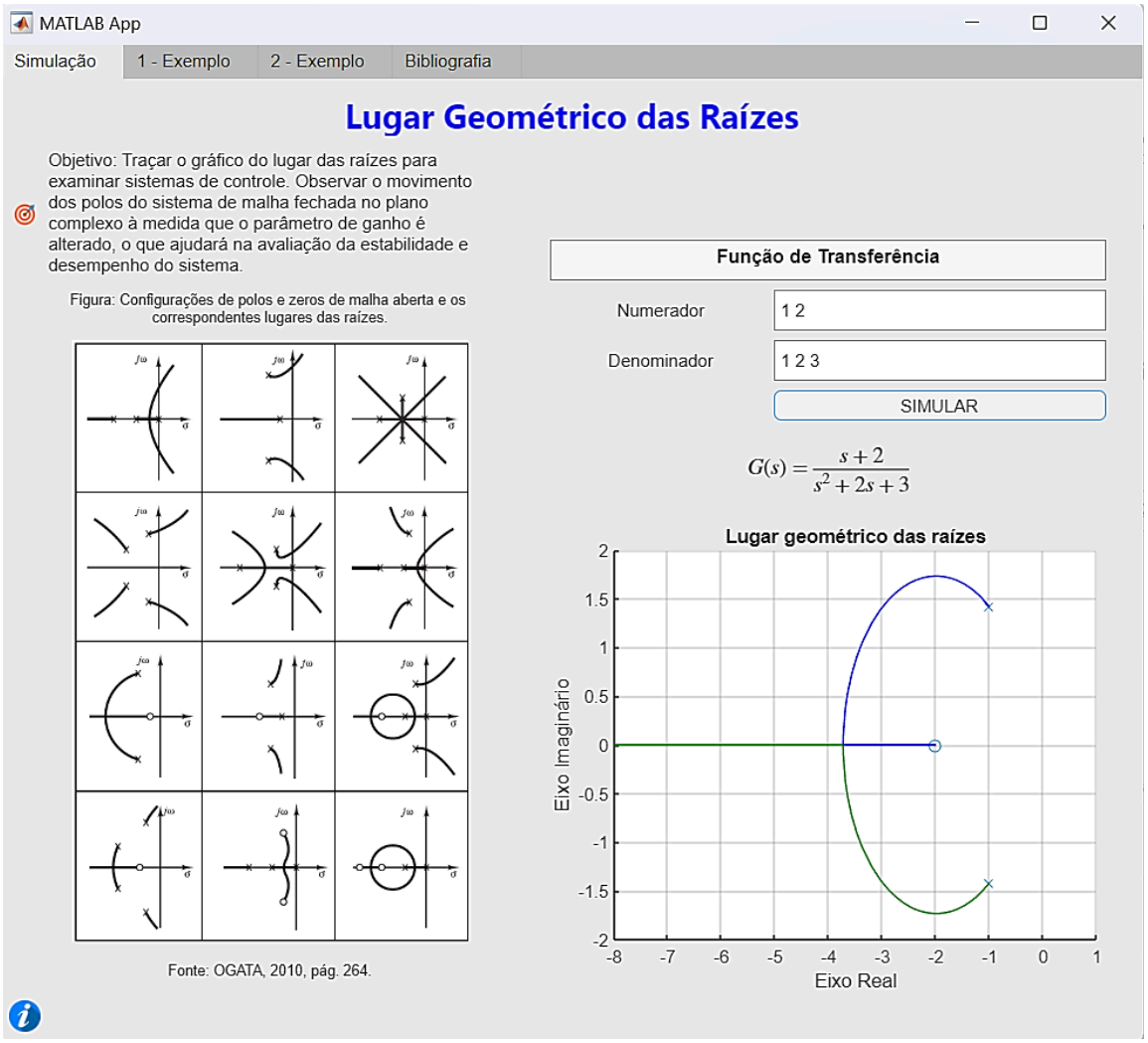
Figura 31: Guia “2 - Exemplo” da Interface Gráfica 3.



Fonte: (Própria, 2024).

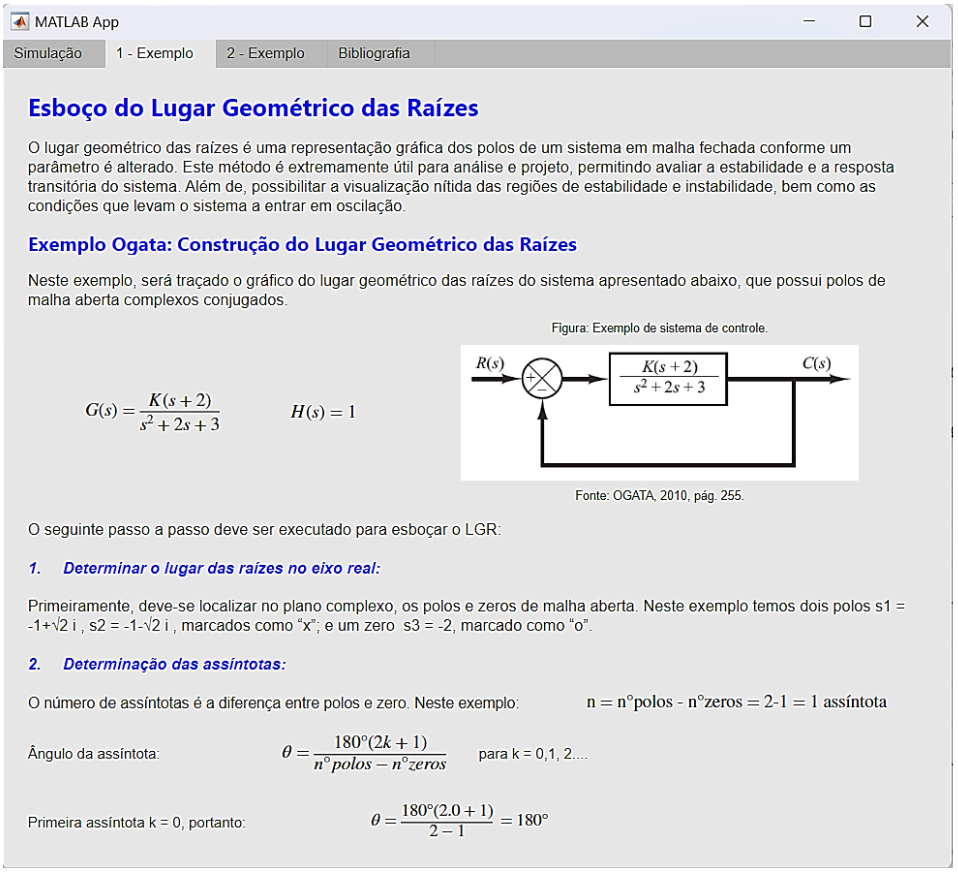
- ***INTERFACE 4: Lugar Geométrico das Raízes***

Figura 32: Guia “Simulação” da Interface Gráfica 4.



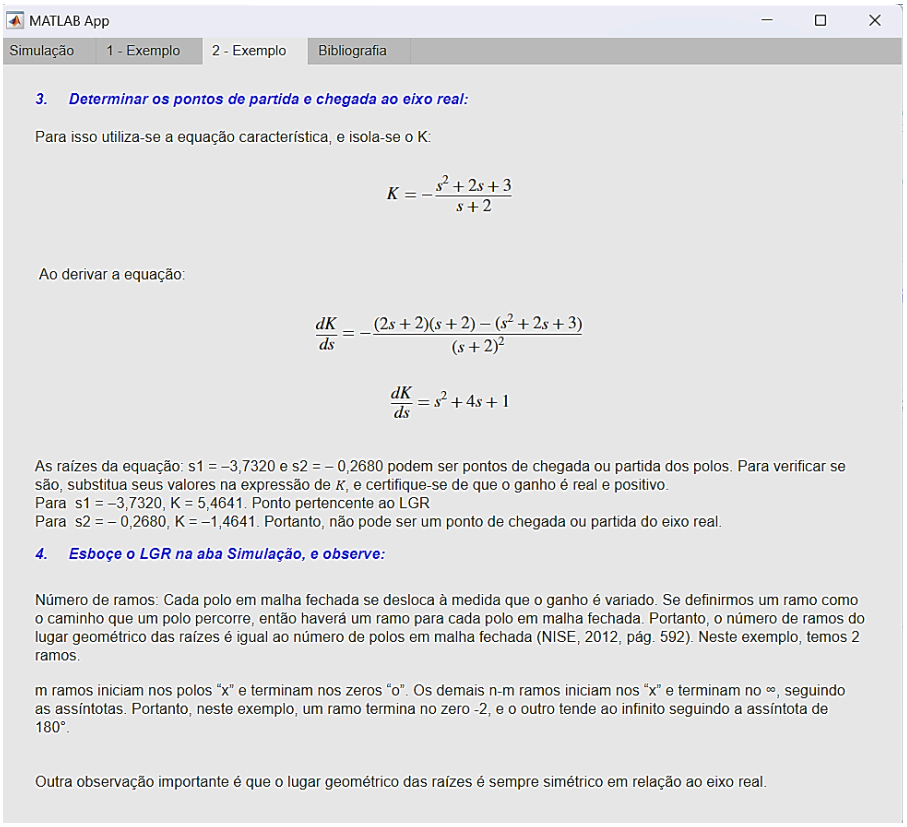
Fonte: (Própria, 2024).

Figura 33: Guia “1 - Exemplo” da Interface Gráfica 4.



Fonte: (Própria, 2024).

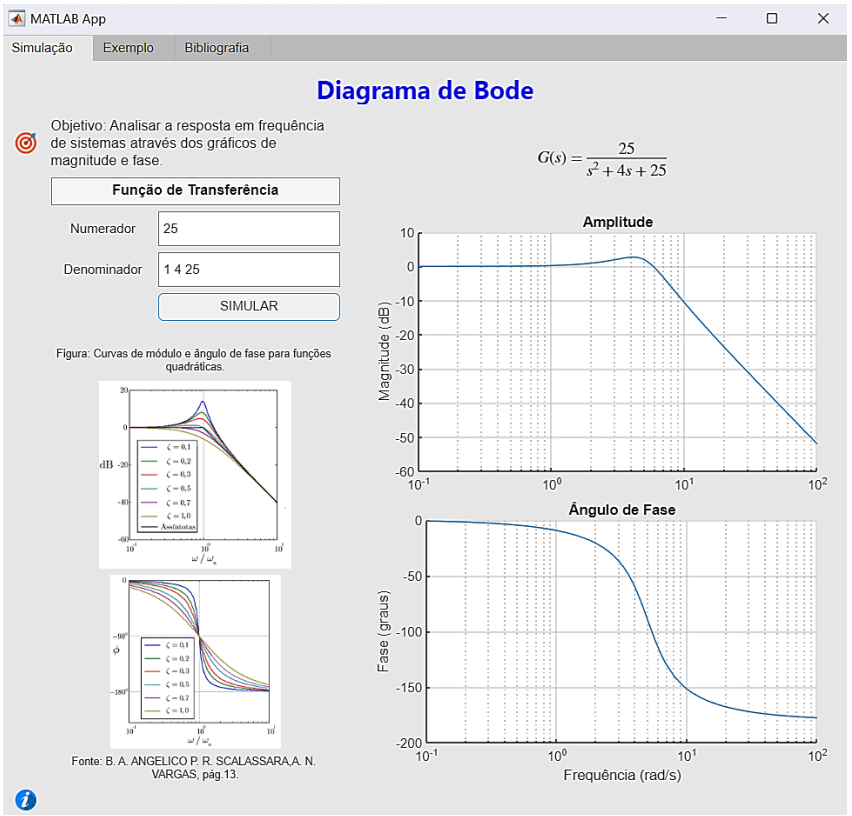
Figura 34: Guia “2 - Exemplo” da Interface Gráfica 4.



Fonte: (Própria, 2024).

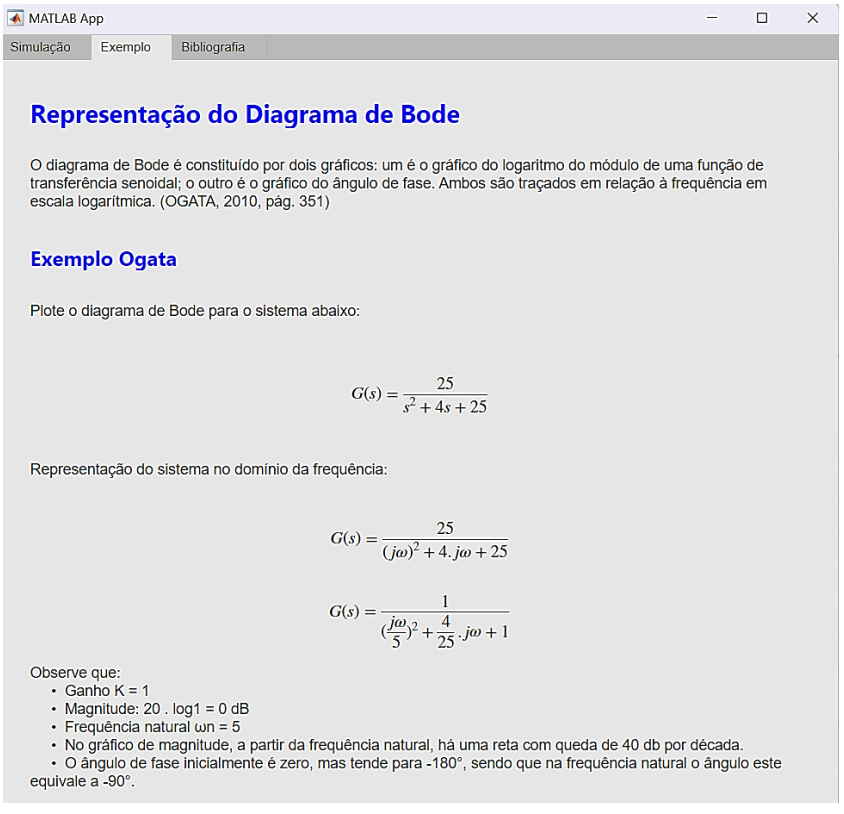
- ***INTERFACE 5: Diagrama de Bode***

Figura 35: Guia “Simulação” da Interface Gráfica 5.



Fonte: (Própria, 2024).

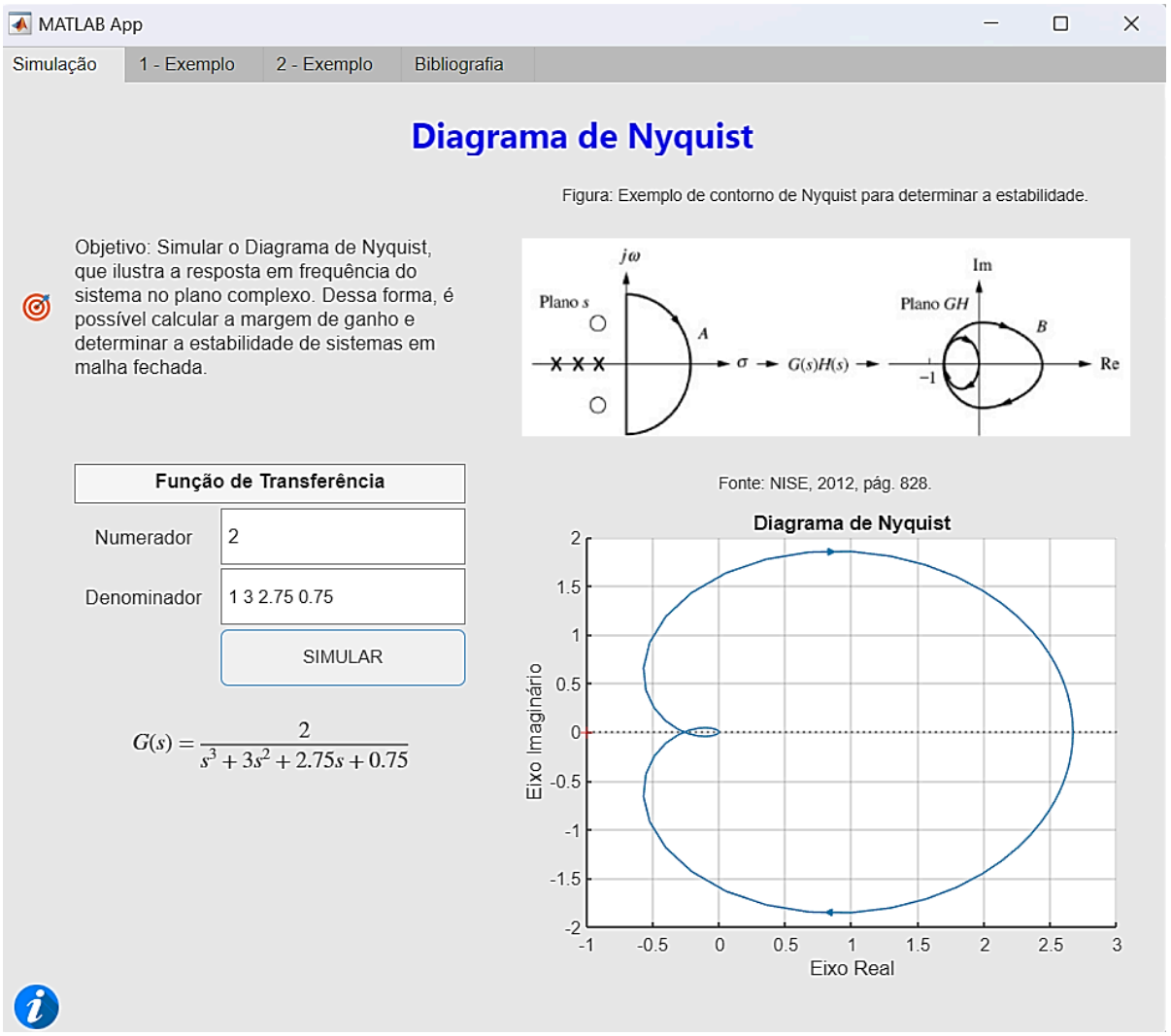
Figura 36: Guia “Exemplo” da Interface Gráfica 5.



Fonte: (Própria, 2024).

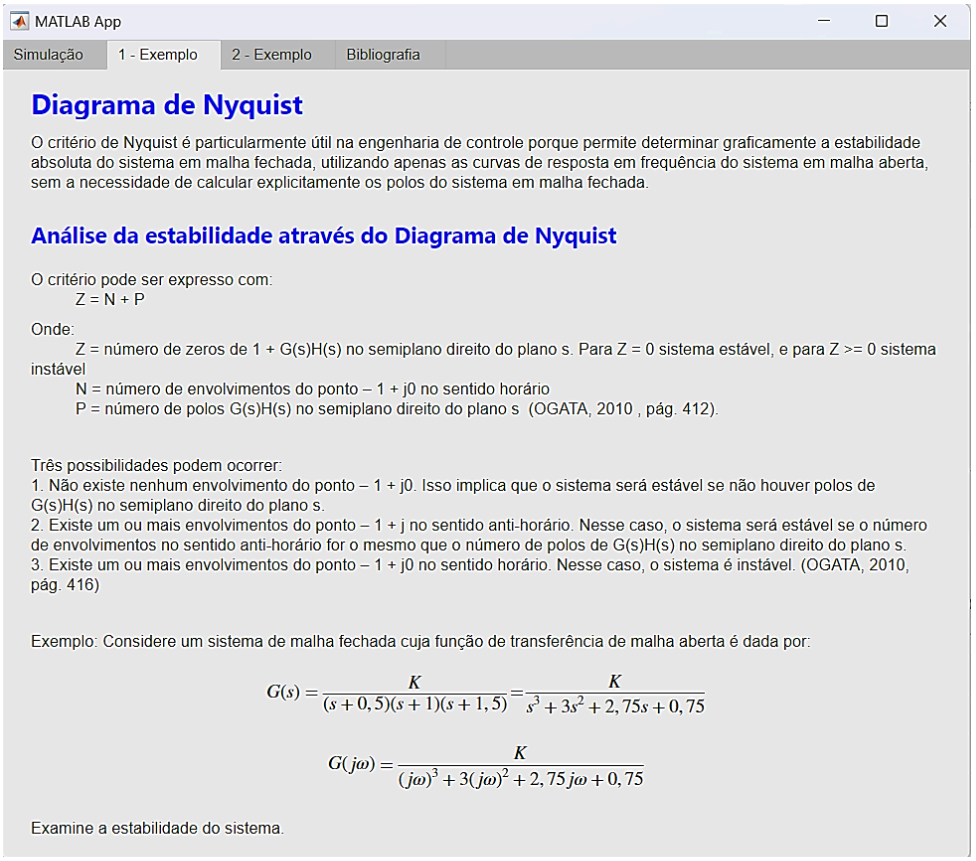
- ***INTERFACE 6: Diagrama de Nyquist***

Figura 37: Guia “Simulação” da Interface Gráfica 6.



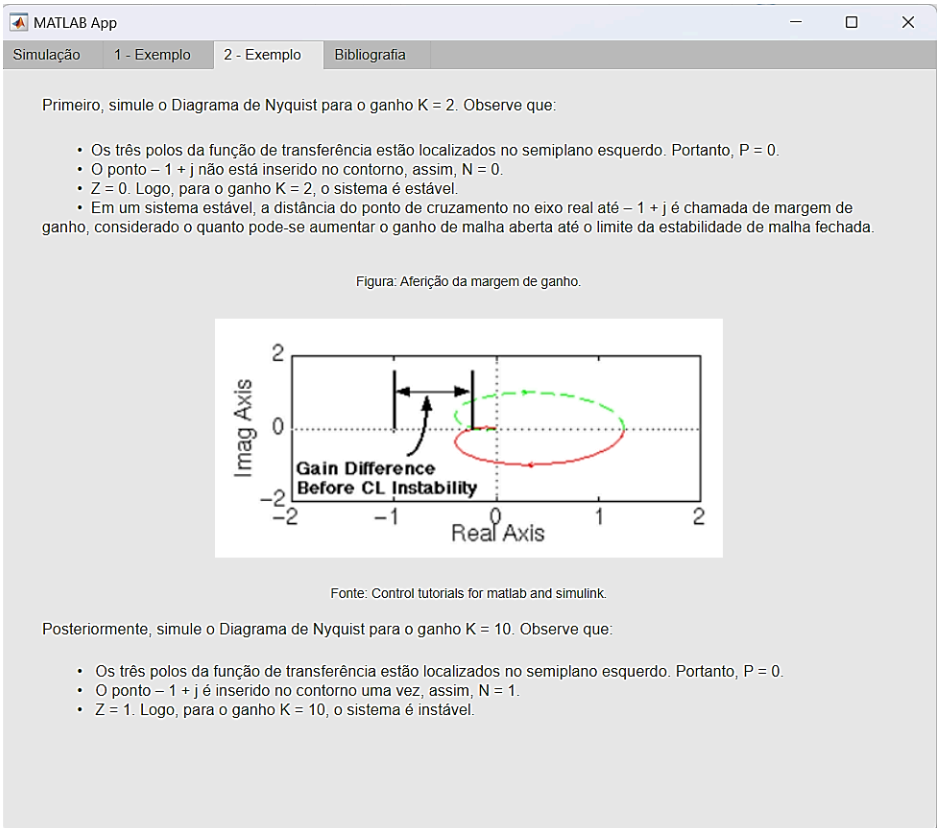
Fonte: (Própria, 2024).

Figura 38: Guia “1 - Exemplo” da Interface Gráfica 6.



Fonte: (Própria, 2024).

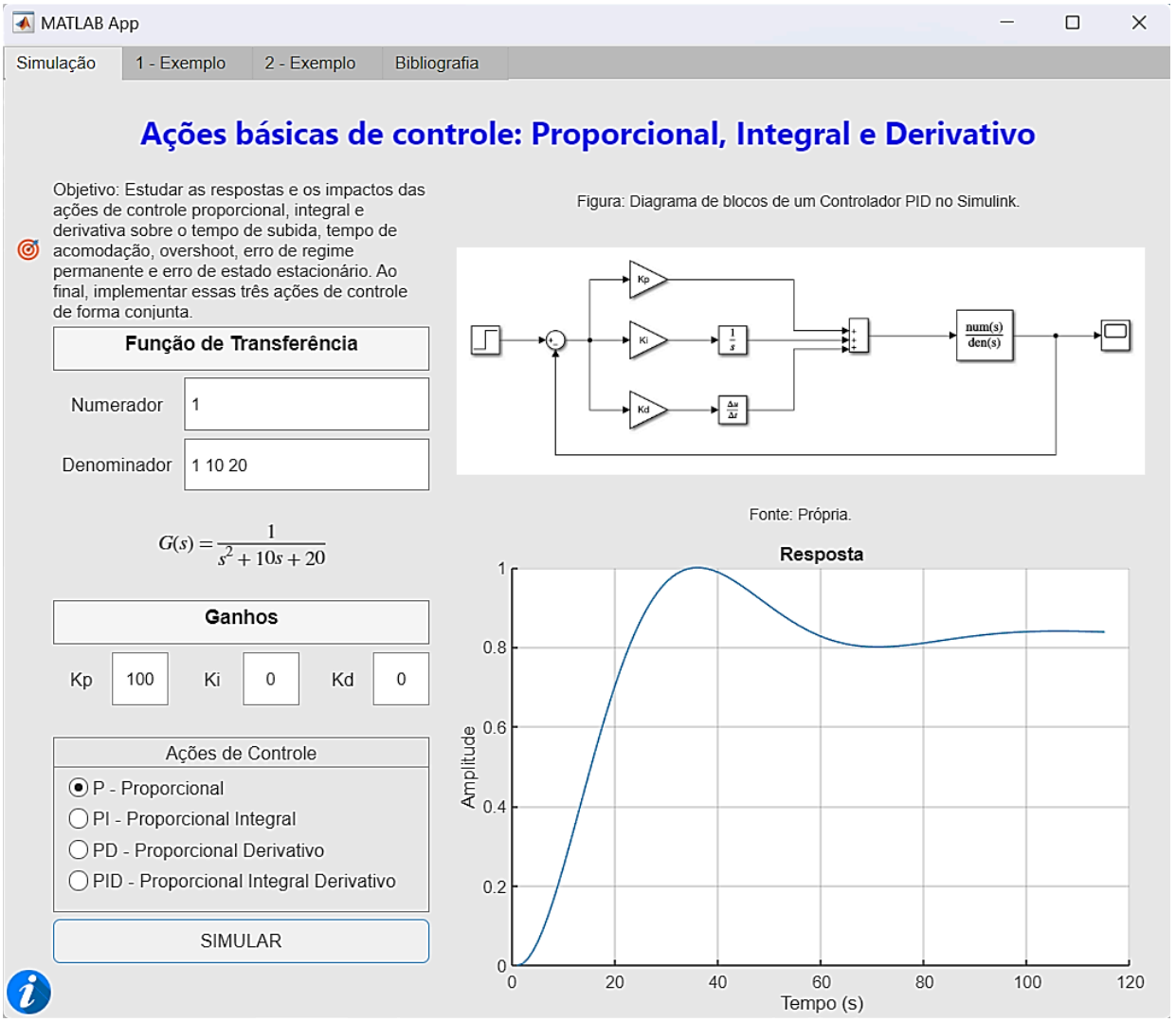
Figura 39: Guia “2 - Exemplo” da Interface Gráfica 6.



Fonte: (Própria, 2024).

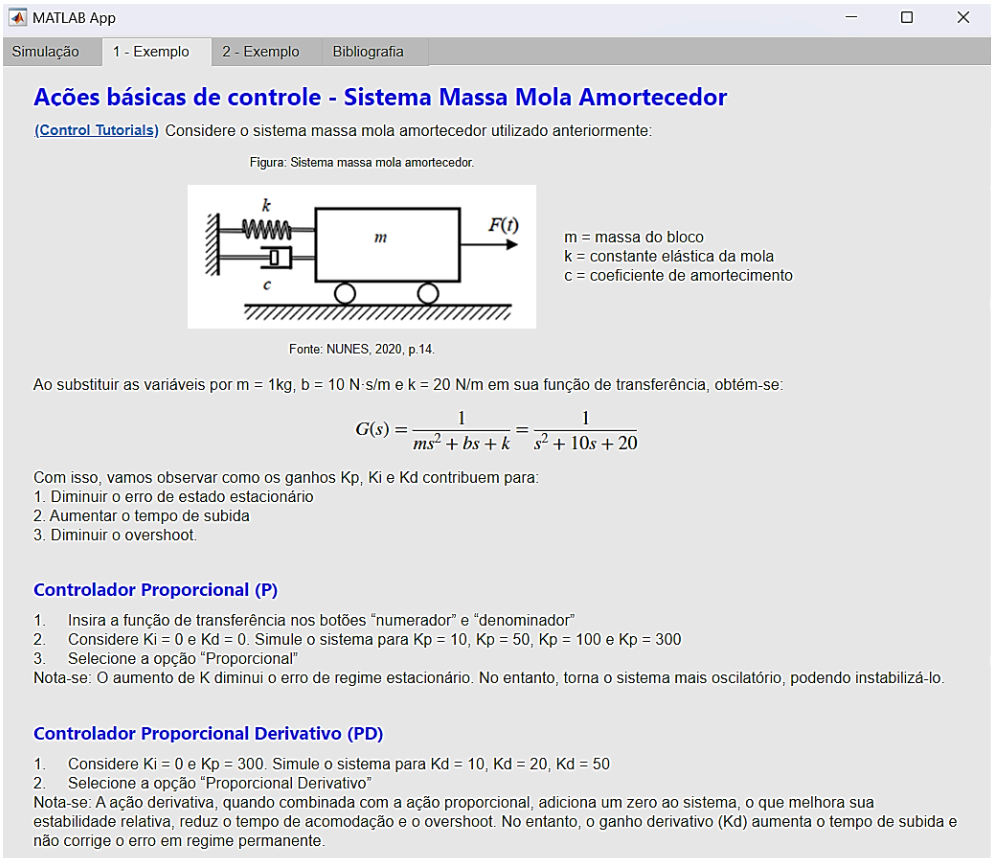
- ***INTERFACE 7: Ações básicas de controle: Proporcional, Integral e Derivativo***

Figura 40: Guia “Simulação” da Interface Gráfica 7.



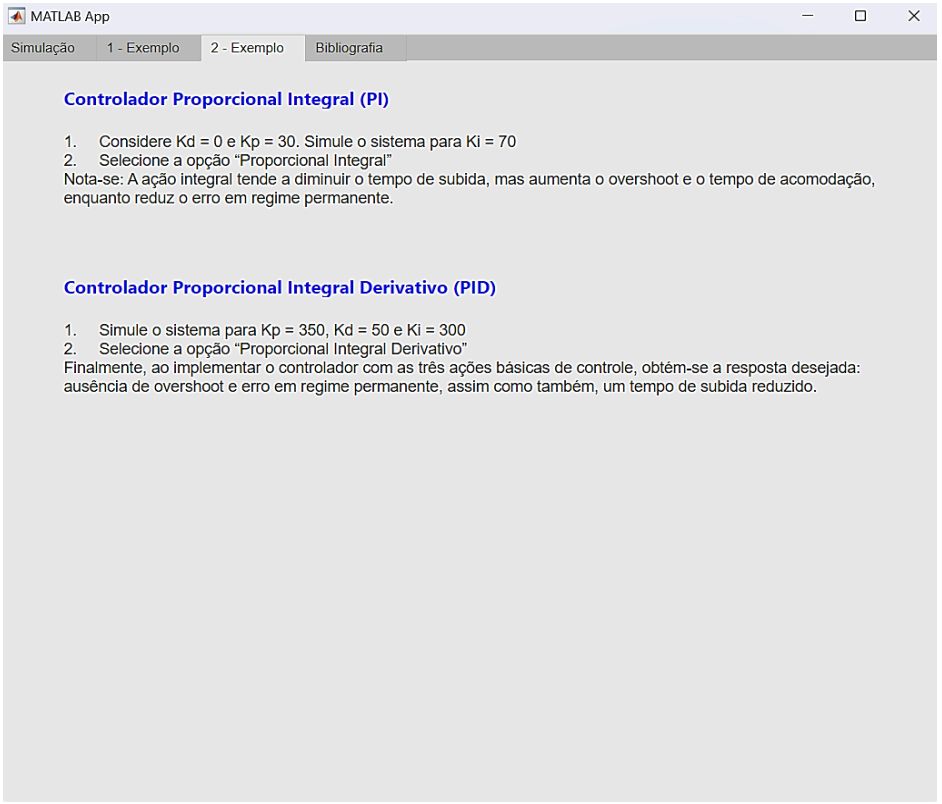
Fonte: (Própria, 2024).

Figura 41: Guia “1 - Exemplo” da Interface Gráfica 7.



Fonte: (Própria, 2024).

Figura 42: Guia "2 - Exemplo" da Interface Gráfica 7.

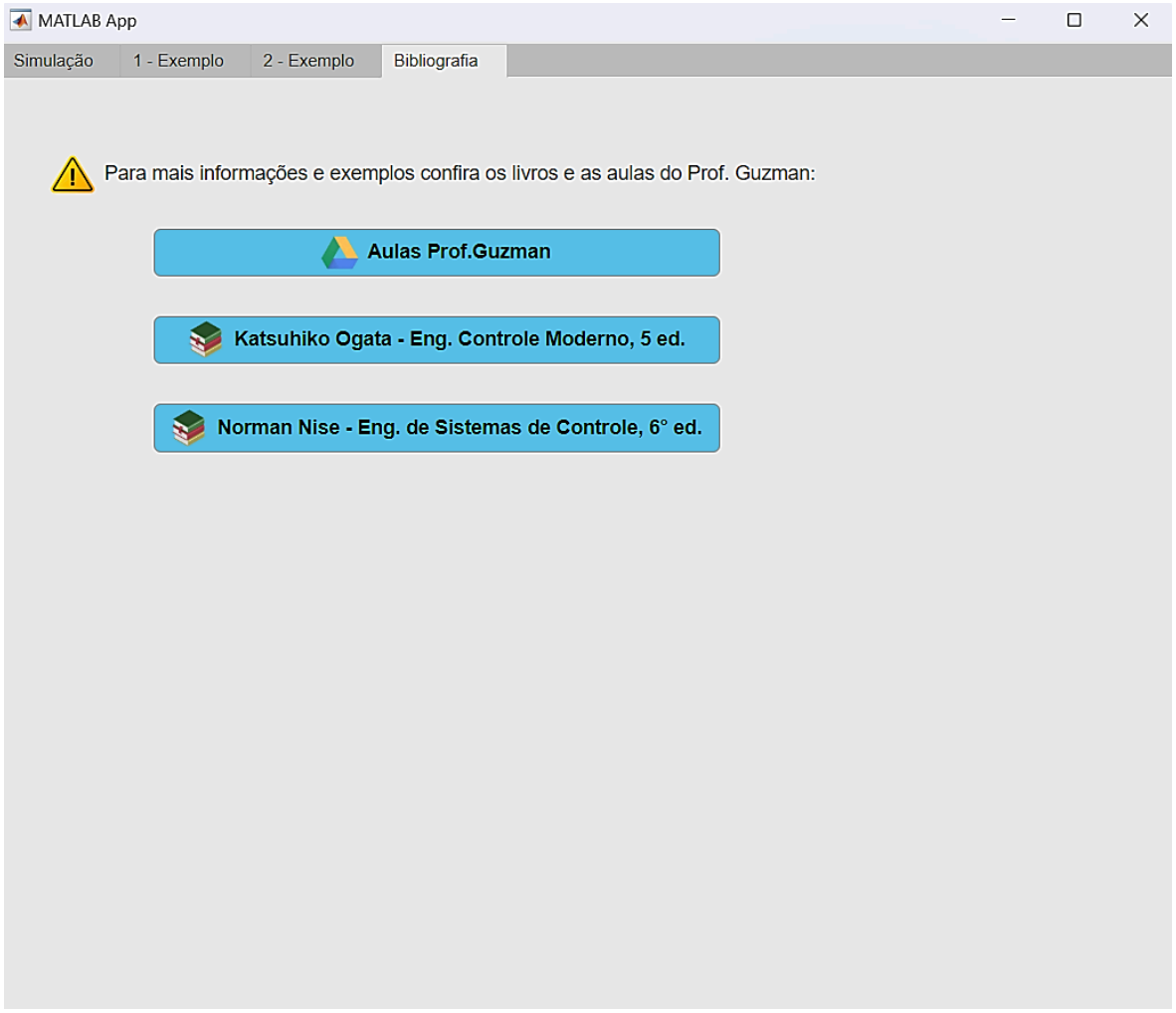


Fonte: (Própria, 2024).

- ***GUIA "Bibliografia"***

Essa guia está presente com o mesmo design em todas as sete interfaces gráficas. Ela é composta por três botões, com o tipo de *callback* *ButtonPushedFcn*, que utiliza o comando *winopen* para abrir os PDFs dos respectivos capítulos dos livros de Ogata e Nise, e o comando *web* para acessar o drive de videoaulas do professor Guzman. O objetivo é incentivar os alunos a aprofundarem seus estudos sobre os assuntos de sistema de controle, e encontrarem outros exercícios para simulação.

Figura 43: Guia “Bibliografia” presente nas 7 Interfaces.



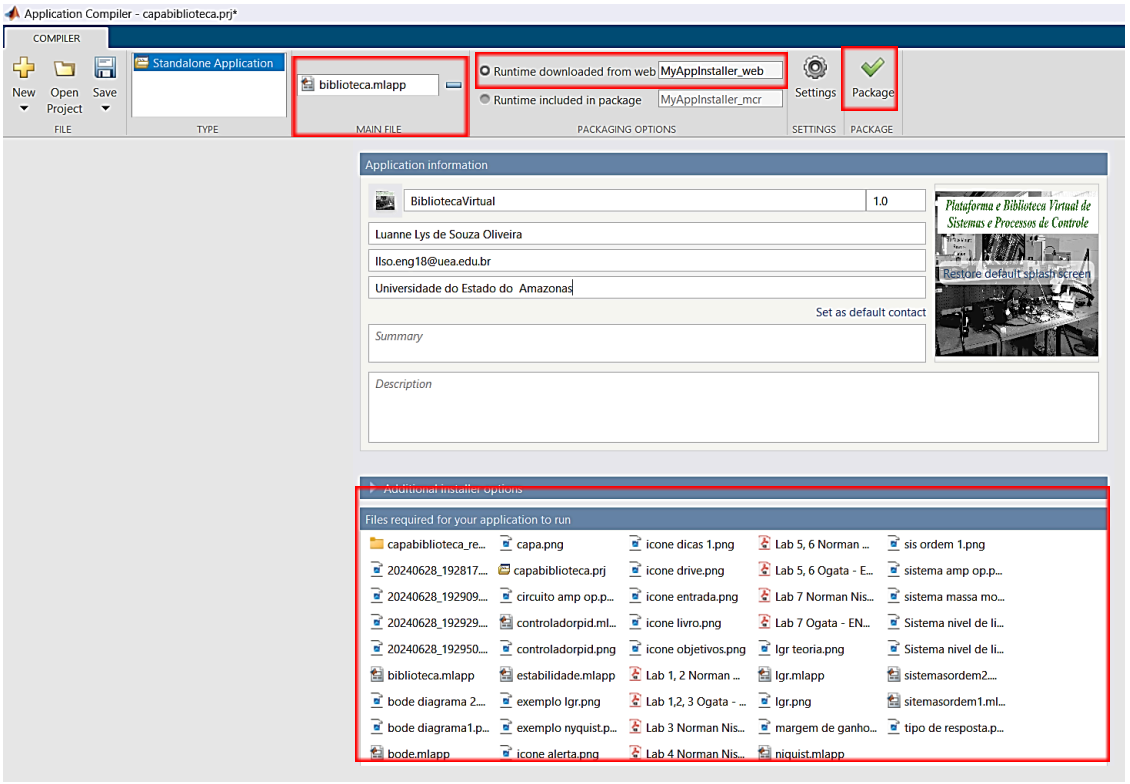
Fonte: (Própria, 2024).

3.5 Compilação e instalação da Biblioteca Virtual

A biblioteca é compilada como um aplicativo desktop, na opção "*Standalone Desktop App",* no botão "*Share"* do App Designer. A aba de compactação é exibida na figura 44, onde o usuário deve-se identificar, selecionar o arquivo principal (neste caso, a capa da biblioteca), escolher um ícone para o aplicativo, e uma imagem que será mostrada durante o processo de instalação. Na seção, "*files required for your application to run"*, é necessário incluir todas as imagens, PDFs e arquivos .mlapp utilizados no desenvolvimento das interfaces.

Ao selecionar a opção "*Runtime downloaded from web"*, o aplicativo é compilado com um arquivo *runtime* que inclui todas as bibliotecas, serviços e recursos necessários para a execução da plataforma no ambiente de destino. Essa configuração possibilita que usuários sem o Matlab instalado em seus computadores, possam utilizar normalmente a Biblioteca Virtual de Sistemas de Controle.

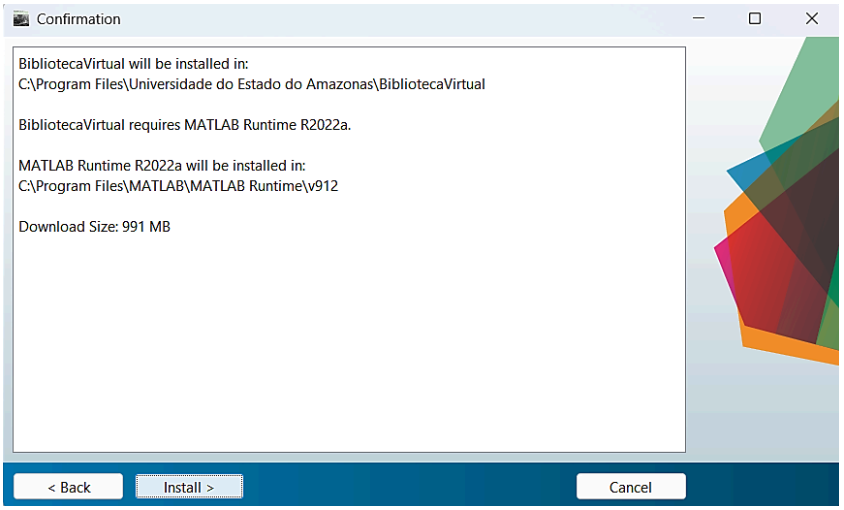
Figura 44: Guia de compilação do aplicativo Desktop.



Fonte: (Própria, 2024).

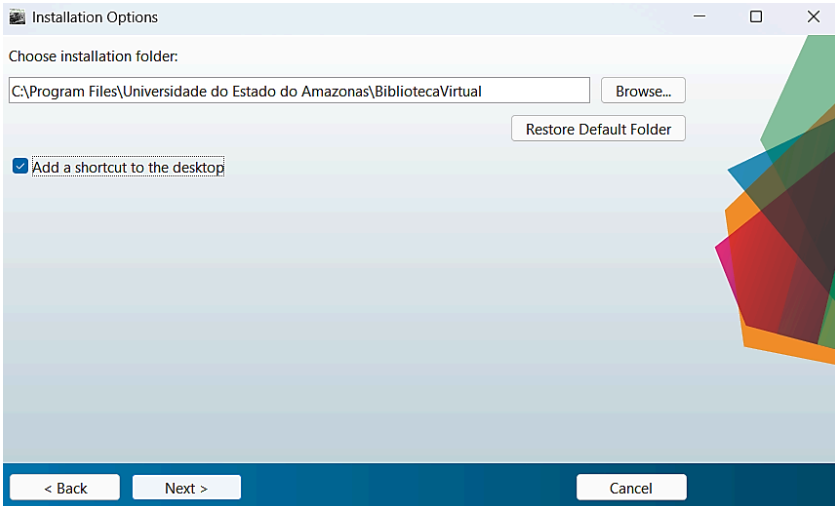
Para instalar a biblioteca, é necessário possuir apenas o arquivo executável "*MyAppInstaller_web.exe"* e seguir as instruções fornecidas, algumas são mostradas nas figuras 45 e 46.

Figura 45: Arquivos instalados para a execução do app com tamanho especificado de 991MB.



Fonte: (Própria, 2024).

Figura 46: Localização do arquivo principal e opção de atalho na área de trabalho.



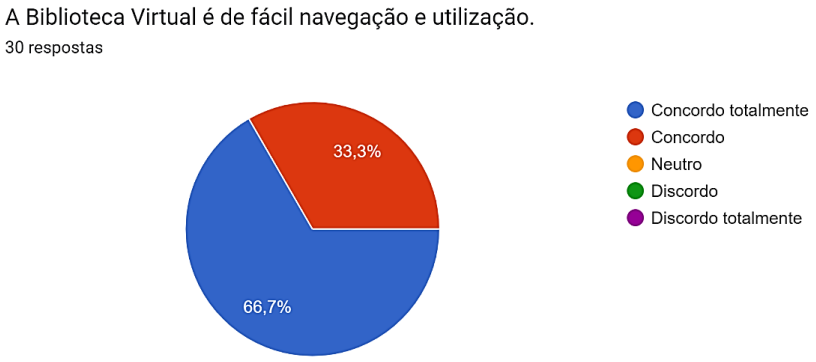
Fonte: (Própria, 2024).

4. PESQUISA DE ACEITAÇÃO

Durante duas aulas nos dias 26/06/2024 e 01/07/2024, a Biblioteca Virtual foi apresentada para os estudantes da disciplina de Sistemas de Controle I do curso de Engenharia Elétrica, onde simulações foram realizadas principalmente na Interface 7, para entender os efeitos das ações básicas de controle.

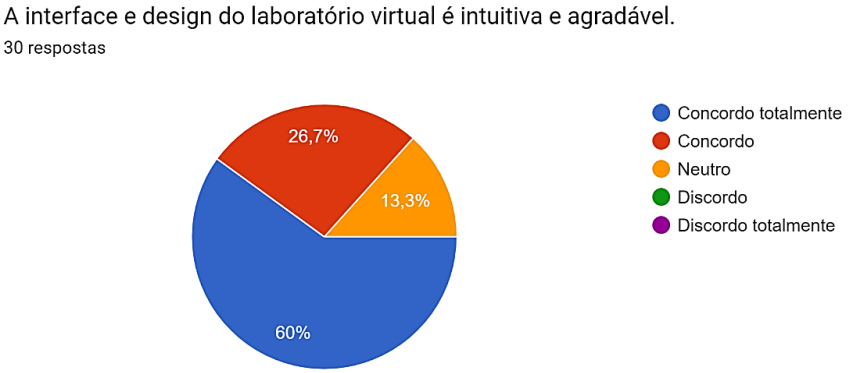
Após as aulas, foi enviado um questionário por meio do Google Formulários aos estudantes, com a finalidade de avaliar a sua experiência na plataforma. Ao todo, 30 estudantes responderam sobre a interatividade, usabilidade, navegação e design da biblioteca virtual. O questionário incluía uma pergunta sobre a recomendação da biblioteca, e cinco afirmações para as quais os usuários indicaram o seu grau de concordância, com opções de resposta que variavam de Concordo Totalmente a Discordo Totalmente. As respostas obtidas são apresentadas abaixo:

Figura 47: Afirmação “A Biblioteca Virtual é de fácil navegação e utilização”.



Fonte: (Própria, 2024).

Figura 48: Afirmação “A interface e design do laboratório virtual é intuitiva e agradável”.



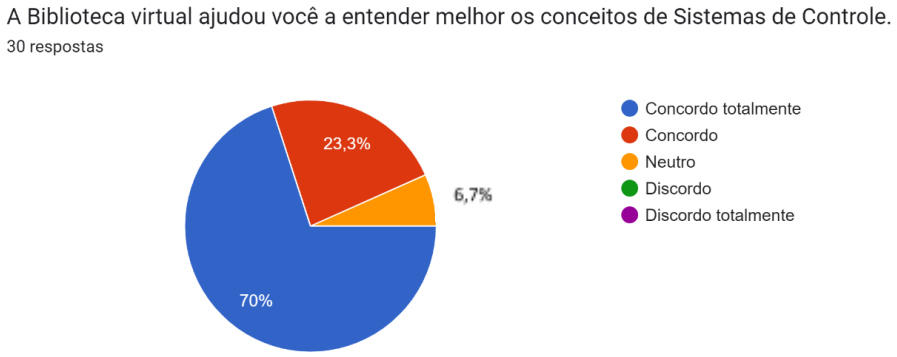
Fonte: (Própria, 2024).

Figura 49: Afirmação “Ótimos recursos disponíveis para simulações e interatividade”.



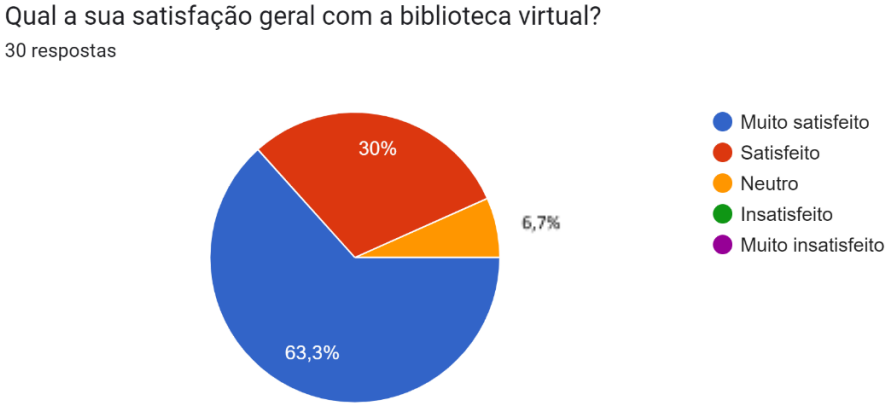
Fonte: (Própria, 2024).

Figura 50: Afirmação “A Biblioteca virtual ajudou você a entender melhor os conceitos de Sistemas de Controle”.



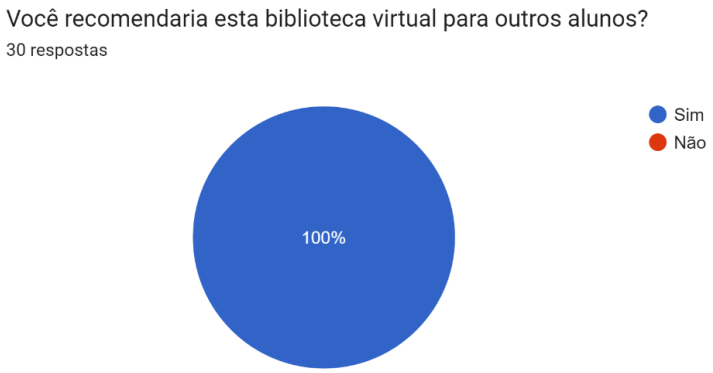
Fonte: (Própria, 2024).

Figura 51: Afirmação “Qual a sua satisfação geral com a biblioteca virtual?”.



Fonte: (Própria, 2024).

Figura 52: Pergunta “Você recomendaria esta biblioteca virtual para outros alunos?”.



Fonte: (Própria, 2024).

Os resultados da pesquisa revelam que a maioria dos estudantes avaliou positivamente a biblioteca virtual de sistemas de controle. Ao todo, 64,66% concordaram totalmente com as afirmações; 28% concordaram com as afirmações; e 7,34% se mantiveram neutros. Assim como também, todos os participantes afirmaram que recomendariam a biblioteca a outros colegas.

A interatividade, usabilidade e navegação receberam avaliações positivas, com mais da metade dos estudantes concordando totalmente ou concordando com as declarações sobre esses aspectos. Isso demonstra a eficácia da ferramenta e recursos disponíveis, que visavam facilitar o aprendizado e a compreensão dos conceitos de sistemas de controle.

O design e layout também receberam avaliações positivas, 60% dos participantes marcaram concordo totalmente, e 26,7% marcaram concordo. Durante as aulas, os estudantes elogiaram a estética e a clareza visual da plataforma, destacando a organização dos conteúdos como um ponto forte. Desta forma, o design da biblioteca foi bem recebido.

No entanto, alguns estudantes responderam pesquisa de aceitação de forma neutra, possivelmente porque precisam de mais tempo utilizando a biblioteca para identificar pontos de melhoria e fornecer sugestões mais detalhadas e pertinentes.

Contudo, a finalidade desta pesquisa de aceitação era obter a primeira impressão dos usuários. A princípio, os resultados indicam que a biblioteca virtual de sistemas de controle atende às necessidades dos estudantes, especialmente nos aspectos de interatividade, navegação e design. E a aceitação positiva sugere que a plataforma é um recurso valioso no aprendizado de sistemas de controle.

5. CONCLUSÃO

O trabalho foi fundamental para desenvolver uma plataforma virtual de sistemas de controle no App Designer, destinada a estudantes de engenharia elétrica e eletrônica. Dessa forma, ofereceu-se uma nova opção de metodologia de ensino, por meio de um aplicativo desktop, que visa aprimorar as habilidades técnicas e comportamentais dos alunos.

A análise indica que a biblioteca virtual é uma ferramenta valiosa para auxiliar os alunos no aprendizado de sistemas de controle, ao permitir uma compreensão mais interativa e envolvente dos tópicos. Além disso, a plataforma os incentiva a aprofundarem-se nos conteúdos por conta própria, buscar exemplos e realizar simulações de diferentes sistemas. Com essa abordagem de metodologia ativa, os estudantes podem estudar de maneira mais eficaz e alcançar um progresso significativo.

O projeto também simplificou a aplicação de sistemas de controle, ao integrar os conteúdos ensinados em sala de aula com um ambiente virtual. A acessibilidade foi um fator importante alcançado, pois o aplicativo é leve e pode ser executado em computadores que não têm o Matlab instalado. Para acessá-lo, é necessário apenas o arquivo executável e seguir o processo de instalação intuitivo.

A pesquisa de aceitação revelou uma primeira impressão positiva da biblioteca virtual. Portanto, o trabalho demonstrou a importância de laboratórios e bibliotecas virtuais no aprendizado dos alunos, e com isso, pode-se sugerir que essa abordagem seja estendida e incentivada em outras disciplinas dos cursos de engenharia.

Para trabalhos futuros, recomenda-se acompanhar detalhadamente a curva de aprendizagem dos alunos com a plataforma. Pesquisas de aceitação mais aprofundadas sobre cada interface podem ser realizadas, a fim de que os estudantes forneçam observações e sugestões de melhoria específicas. Esse processo permitirá a manutenção contínua do aplicativo, o aprimoramento de suas funcionalidades e o ajuste do design, para tornar a experiência ainda mais agradável.

6. REFERÊNCIAS BIBLIOGRÁFICAS

AGOSTINHO, Douglas Soares; AGOSTINHO, Tânia Frugiuele Soares. **Metodologias ativas em cursos de engenharia a distância: tecnologias aplicadas na busca da inovação e da qualidade de ensino**. Curitiba, Brazilian Journal of Development, 2022.

ALMEIDA, Heraldo L. S. **Introdução aos Sistemas de Controle.** Rio de Janeiro, UFRJ, 2021.

ARAÚJO, Fábio Meneghetti Ugulino. **SISTEMAS DE CONTROLE**. Natal, Rio Grande do Norte, Universidade Federal do Rio Grande do Norte, 2007.

DE MATHEUCCI, Guilherme Augusto et al. **Análise do laboratório virtual de aprendizagem LabHidra.com aplicado ao ensino de hidráulica**. # Tear: Revista de Educação, Ciência e Tecnologia, v. 10, n. 2, 2021. https://doi.org/10.35819/tear.v10.n2.

E-disciplinas USP. **Apostila – Método do Lugar das Raízes, 2016**. Disponível em: < https://edisciplinas.usp.br/mod/resource/view.php?id=1282910&forceview=1>. Acesso em: 20 de maio de 2024.

E-disciplinas USP. **Apostila sobre PID e Métodos de Sintonia**. Disponível em: < https://edisciplinas.usp.br/pluginfile.php/5123982/mod_resource/content/1/Apostila%20sobre%20PID%20e%20M%C3%A9todos%20de%20Sintonia.pdf>. Acesso em: 20 de maio de 2024.

FILATRO, A.; CAVALCANTI, C. C. **Metodologias Inovativas na educação presencial, a distância e corporativa**. São Paulo: Saraiva Educação, 2018.

LOPES, Pedro Cortez; RANGEL, Rafael Lopez; MARTHA, Luiz Fernando. **O ambiente App Designer do MATLAB**. Rio de Janeiro, PUC RIO, 2023.

Mathworks. **Documentation**. Disponível em: <https://www.mathworks.com/help/index.html?s_tid=CRUX_lftnav>. Acesso em: 10 de junho de 2024.

Mathworks. **App Designer**. Disponível em: <https://www.mathworks.com/help/matlab/ref/appdesigner.html?searchHighlight=app%20designer&s_tid=srchtitle_support_results_1_app%20designer>. Acesso em: 10 de junho de 2024.

Mathworks. **App Building Components**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/choose-components-for-your-app-designer-app.html?searchHighlight=component%20library%20%20app%20designer&s_tid=srchtitle_support_results_2_component%20library%20%20app%20designer>. Acesso em: 7 de junho de 2024.

Mathworks. **Callbacks in App Designer**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/write-callbacks-for-gui-in-app-designer.html>. Acesso em: 02 de julho de 2024.

Mathworks. **Lay Out Apps in App Designer Design View**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/lay-out-apps-in-app-designer.html>. Acesso em: 6 de junho de 2024.

Mathworks. **Reuse Code Using Helper Functions**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/code-and-call-app-functions-in-app-designer.html?searchHighlight=startup%20and%20helper%20functions&s_tid=srchtitle_support_results_4_startup%20and%20helper%20functions>. Acesso em: 03 de junho de 2024.

Mathworks. **Startup Tasks and Input Arguments in App Designer**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/app-designer-startup-function.html?searchHighlight=app%20designer&s_tid=srchtitle_support_results_9_app%2520designer>. Acesso em: 03 de junho de 2024.

Mathworks. **TabGroup Properties**. Disponível em: <https://www.mathworks.com/help/matlab/ref/matlab.ui.container.tabgroup-properties.html?searchHighlight=tab%20group&s_tid=srchtitle_support_results_1_tab%2520group>. Acesso em: 07 de junho de 2024.

Mathworks. **Use Grid Layout Managers in App Designer**. Disponível em: <https://www.mathworks.com/help/matlab/creating_guis/using-grid-layout-managers.html?searchHighlight=grid%20layout&s_tid=srchtitle_support_results_2_grid%2520layout>. Acesso em: 05 de junho de 2024.

NISE, N. S. **Engenharia de Sistemas de Controle.** 6ª edição, LTC, 2012.

OGATA, K. - **Engenharia de Controle Moderno.** 5ª edição, Pearson, 2010.

PEREIRA, Luís Fernando Alves; HAFFNER, José Felipe. **Resposta em Freqüência de Sistemas Lineares - Diagramas de Bode**. Rio Grande do Sul, Pontifícia Universidade Católica do Rio Grande do Sul, Departamento de Engenharia Elétrica, 2018.

Rohde&Schwarz. **Compreender Diagramas de Bode**. Disponível em: <https://www.rohde-schwarz.com/br/produtos/teste-e-medicao/essentials-test-equipment/digital-oscilloscopes/compreender-diagramas-de-bode_254514.html#gallery-6>. Acesso em: 22 de maio de 2024.

SCHMITZ, Anelise; NOGUEIRA, Matheus Lemos; NERVIS, Leandro Olivio. **INTERFACES EM LABORATÓRIO VIRTUAL DE APRENDIZAGEM EM ENGENHARIAS – CARACTERIZAÇÃO FÍSICA DE SOLOS.** Revista de Ensino de Engenharia, 2021.

SUTILI, Felipe Korbus; RAINERE, Iara Alessandra Donati. **Metodologias ativas na formação do engenheiro do século XXI: desafios e reflexões.** Ponta Grossa, Olhar de professor, 2022.

VIDOTTO, Kajiana Nuernberg Sartor; TAROUCO, Liane Margarida Rockenbach; SILVA, Patrícia Fernanda; FARACO, Morgana Nuernberg Sartor. **Scratch nas engenharias: Alunos desenvolvedores de experimentos para laboratórios virtuais.** Rio Grande do Sul, Revista Novas Tecnologias na Educação, 2022.

7. FONTES CONSULTADAS

FERNANDEZ, Alejandro D. Pardo. **Desarrollo de Laboratorios Virtuales en entorno Matlab/Simulink para docencia en control**. Gijon, Universidad de Oviedo, 2023.

JUNIOR, Edward Pinto Pimenta. **Elaboração de ambientes virtuais para auxiliar a automação de práticas laboratóriais de sistemas de controle em engenharia elétrica e eletrônica**. Amazonas, Universidade do Estado do Amazonas, 2023.

Mathworks. **Matlab Products**. Disponível em: <https://www.mathworks.com/?s_tid=gn_logo>. Acesso em: 04 de junho de 2024.

Mathworks. **Virtual-Controls-Laboratory**. Disponível em: <https://www.mathworks.com/matlabcentral/fileexchange/100064-virtual-controls-laboratory?s_tid=mlc_fex_email_activity>. Acesso em: 11 de abril de 2024.

ANEXO A – CÓDIGOS DA BIBLIOTECA VIRTUAL

Interface gráfica 1 – Função *respostasistema(app)*

```
methods (Access = private)

    function respostasistema(app)
        K = app.KSlider.Value;
        Tau = app.TauSlider.Value;
        t = app.Tempo.Value;

        G = tf([K], [Tau, 1]);

        opcaoentrada = app.ENTRADAButtonGroup.SelectedObject;

        if opcaoentrada == app.DEGRAUButton
            intervalo = 0:0.1:(t/10); % Corrige a escala do eixo x, vetor de tempo de 0 a t segundos
            % com tempo de amostragem 0.1
            x = step(G, intervalo);
            plot(app.UIAxes, x);
            title(app.UIAxes, 'Resposta ao Degrau');
            axis auto; % atualiza o gráfico a cada simulação
        end

        if opcaoentrada == app.RAMPAButton
            intervalo = 0:0.01:t;  % Vetor de tempo de 0 a t segundos com tempo de amostragem 0.1
            rampa = intervalo; % Vetor rampa
            [y, t, x] = lsim(G, rampa, intervalo); % Simulação da resposta do sistema
            plot(app.UIAxes,t, y);
            title(app.UIAxes, 'Resposta a Rampa');
            axis auto; % atualiza o gráfico a cada simulação

        end

    end
end
```

Interface gráfica 2 – Função *respostasistema2(app)*

```
methods (Access = private)

    function respostasistema2(app)
        Wn = app.WnSlider.Value;
        Amortec = app.AmorSlider.Value;
        t = app.TempoSlider.Value;

        G = tf(Wn*Wn, [1, 2*Amortec*Wn, Wn.^2]);

        opcaoentrada = app.ENTRADAButtonGroup.SelectedObject;

        if opcaoentrada == app.DEGRAUButton
            intervalo = 0:0.1:t/10; % Corrige a escala do eixo x, tempo de amostragem
            a = step(G, intervalo);
            plot(app.UiAxes, a);
            title(app.UiAxes, 'Resposta ao Degrau');

        end

        if opcaoentrada == app.RAMPAButton
            intervalo = 0:0.01:t;  % Vetor de tempo de 0 a 10 segundos com passo de 0.01
            rampa = intervalo; % Vetor rampa
            [y, t, x] = lsim(G, rampa, intervalo); % Simulação da resposta do sistema
            plot(app.UiAxes, t, y);
            title(app.UiAxes, 'Resposta a Rampa');

        end
    end
end
```

Interface gráfica 3 – Função *resposta(app)*

```
methods (Access = private)


    function resposta(app)
        numerador = str2num(app.NumeradorEditField.Value);
        denominador = str2num(app.DenominadorEditField.Value); % converte os valores inseridos para vetor

        G = tf(numerador, denominador);

        % Formatando a função de transferência para inserir no componente Label
        [num, den] = tfdata(G, 'v');
        numconv = poly2str(num, 's');
        denconv = poly2str(den, 's');
        equacaolatex = sprintf('$$G(s) = \\frac{%s}{%s}$$', numconv, denconv);

        app.Tf.Text = equacaolatex;


        % Gráfico resposta ao degrau
        a = step(G);
        plot(app.ResAxes, a);


        % Código para exibir os polos e zeros no UiAxes
        [z, p, k] = zpkdata(G, 'v'); % 'v' define como vetor, separa os polos e zeros da função
        cla(app.PosAxes); %limpa o grafico, remove as informações da função anterior
        plot(app.PosAxes, real(z), imag(z), 'ob'); % Plota os Zeros com marcadores azuis
        hold(app.PosAxes, 'on'); % Espera para plotar mais informações no mesmo grafico,
        % mantém o gráfico atual para adicionar os polos
        plot(app.PosAxes, real(p), imag(p), 'xr'); % Plota os polos com marcadores vermelhos


    end

end
```

Interface gráfica 4 – Função *lugargeometrico(app)*

```
methods (Access = private)

    function lugargeometrico(app)
        numerador = str2num(app.NumeradorEditField.Value);
        denominador = str2num(app.DenominadorEditField.Value); %converte os valores inseridos para vetor

        G = tf(numerador, denominador);

        % Formatando a função de transferência para inserir no componente Label
        [num, den] = tfdata(G, 'v');
        numconv = poly2str(num, 's');
        denconv = poly2str(den, 's');
        equacaolatex = sprintf('$$G(s) = \\frac{%s}{%s}$$', numconv, denconv);

        app.Tf.Text = equacaolatex;



        cla(app.LgrAxes); %limpa o gráfico, remove as informações da função anterior

        f = figure; %Cria uma figura para plotar o lgr
        rlocus(G);
        lgr = findobj(f, 'Type', 'Axes'); %Procura pelos objetos do gráfico gerado pelo rlocus na figure f
        copyobj(lgr.Children, app.LgrAxes); %Insere as propriedades no LgrAxes
        app.LgrAxes.XLim = lgr.XLim; %Copia a escala do eixo x
        app.LgrAxes.YLim = lgr.YLim; %Copia a escala do eixo y

        delete(f) % Apaga a figura gerada inicialmente

    end

end
```

Interface gráfica 5 – Função *Bode(app)*

```
methods (Access = private)

    function Bode(app)
        numerador = str2num(app.NumeradorEditField.Value);
        denominador = str2num(app.DenominadorEditField.Value); % converte os valores inseridos para vetor

        G = tf(numerador, denominador);


        % Formatando a função de transferência para inserir no componente Label
        [num, den] = tfdata(G, 'v');
        numconv = poly2str(num, 's');
        denconv = poly2str(den, 's');
        equacaolatex = sprintf('$$G(s) = \\frac{%s}{%s}$$', numconv, denconv);

        app.Tf.Text = equacaolatex;


        %  Obter dados do diagrama de Bode
        [mag, phase, w] = bode(G);

        % Converter magnitude
        mag_amp = squeeze(mag);
        % Plotar somente a amplitude
        cla(app.AmplitudeAxes);  %limpa o gráfico, remove as informações da função anterior
        fa = figure;
        semilogx(w, 20*log10(mag_amp));
        a = findobj(fa, 'Type', 'Axes');
        copyobj(a.Children, app.AmplitudeAxes); %Insere as propriedades no Axes
        delete(fa);
```

```
        % Converter fase para graus
        fase_deg = squeeze(phase);
        % Plotar somente a fase
        cla(app.FaseAxes);  %limpa o gráfico, remove as informações da função anterior
        ff = figure;
        semilogx(w, fase_deg);
        f = findobj(ff, 'Type', 'Axes');
        copyobj(f.Children, app.FaseAxes); %Insere as propriedades no Axes
        delete(ff);


    end
end
```

Interface gráfica 6 – Função *nyquist(app)*

```
    methods (Access = private)

        function nyquist(app)
            numerador = str2num(app.NumeradorEditField.Value);
            denominador = str2num(app.DenominadorEditField.Value); % converte os valores inseridos para vetor

            G = tf(numerador, denominador);


            % Formatando a função de transferência para inserir no componente Label
            [num, den] = tfdata(G, 'v');
            numconv = poly2str(num, 's');
            denconv = poly2str(den, 's');
            equacaolatex = sprintf('$$G(s) = \\frac{%s}{%s}$$', numconv, denconv);

            app.Tf.Text = equacaolatex;



            cla(app.NyquistAxes);  % Limpa os eixos antes de plotar
            f = figure; %Cria uma figura para plotar o lgr
            nyquist(G);
            n = findobj(f, 'Type', 'Axes');
            copyobj(allchild(n), app.NyquistAxes); %Insere dentro do axes
            %grid on; verificar
            delete(f)

        end
    end
```

Interface gráfica 7 – Função *controlador(app)*

```
methods (Access = private)

    function controlador(app)

        Kp = app.KpEditField.Value;
        Ki = app.KiEditField.Value;
        Kd = app.KdEditField.Value;

        numerador = str2num(app.NumeradorEditField.Value);
        denominador = str2num(app.DenominadorEditField.Value); % converte os valores inseridos para vetor

        G = tf(numerador, denominador);
        s = tf('s');

        % Formatando a função de transferência para inserir no componente Label
        [num, den] = tfdata(G, 'v');
        numconv = poly2str(num, 's');
        denconv = poly2str(den, 's');
        equacaolatex = sprintf('$$G(s) = \\frac{%s}{%s}$$', numconv, denconv);

        app.Tf.Text = equacaolatex;



        selectedButton = app.AesdeControleButtonGroup.SelectedObject;

        if selectedButton == app.PProporcionalButton
            c = Kp;
        end


        if selectedButton == app.PIProporcionalIntegralButton
            c = Kp + Ki/s;
        end

        if selectedButton == app.PDProporcionalDerivativoButton
            c = Kp + Kd*s;
        end


        if selectedButton == app.PIDProporcionalIntegralDerivativoButton
            c = Kp + Ki/s + Kd*s;
        end


        sistema = feedback(c*G,1);

        x = step(sistema);
        plot(app.UIAxes, x);


    end
end
```

ANEXO B – DICAS PARA SIMULAÇÕES NA BIBLIOTECA VIRTUAL

Textos das dicas

Interfaces gráficas 1 e 2

1 - Use um tempo de simulação longo (ex: 500 s) para a entrada degrau, assim, não haverá problemas com a escala do gráfico. Em alguns casos, a resposta pode não aparecer por completo devido ao baixo tempo de simulação.

2 - Use um tempo de simulação curto (ex: 20 s) para a entrada rampa.

3 - Se você ajustar o zoom ou modificar qualquer propriedade do gráfico, após simular outro sistema novamente, clique no ícone da 'casa' para restaurar a escala original.

Interface gráfica 3

1 - Nos botões “Numerador” e “Denominador” você pode inserir os comandos das seguintes maneiras:

- [1 2 3]
- 1.5 2 3
- 1.5 , 2, 3
- 0 2 3 = s^3 + s + 3

2 - A função step não simula a resposta de uma função de transferência com o numerador de grau maior que o denominador.

3 - Se você ajustar o zoom ou modificar qualquer propriedade do gráfico, após simular outro sistema, clique no ícone da 'casa' para restaurar a escala original.

Interfaces gráficas 4, 5, 6 e 7

1 - Nos botões “Numerador” e “Denominador” você pode inserir os comandos das seguintes maneiras:

- [1 2 3]
- 1.5 2 3
- 1.5, 2, 3
- 0 2 3 = s^3 + s + 3

2 - Se você ajustar o zoom ou modificar qualquer propriedade do gráfico, após simular outro sistema, clique no ícone da 'casa' para restaurar a escala original.